**סמינר בית יעקב**
בהנהלת הגב' י. גברא תחי'
כתובת: הרב כהנמן 100 ב"ב | טל: 03-5794002 | פקס: 03-7731105

# פרויקט מערכת מדידת חלל אוטונומית




מגישה: תמר מאזוז
ת"ז: 327821054
סמל מוסד: 595223
תאריך הגשה: 17/06/2026
מורה מנחה: אביה גברא
מכללה: סמינר גברא בית יעקב

# תוכן עניינים

הצעת פרויקט משובצות.....3
תקציר: .....7
תצהיר חתום:.....8
מושגים: .....9
תיאור הנושא: .....10
תיאור הבעיה: .....10
ניתוח חלופות מערכתי: .....11
תיאור החלופה הנבחרת:.....13
הסבר על הבקר:.....14
הסבר על חיישנים:.....16
תיאור הארכיטקטורה:.....22
תיאור פרוטוקולי תקשורת: .....23
הסבר על הספריות: .....24
תהליך הפיתוח:.....28
תיעוד הפתרון: .....29
שרטוט חשמלי:.....29
חיבורים ואופן הפעולה: .....30
קטעי קוד:.....32
Motors:.....34
Clock:.....35
Laser_sensor: .....36
דיאגרמת בלוקים: .....37
תרשים זרימה: .....37
GUI: .....38
פיתוחים עתידיים ושדרוגי מערכת: .....39
סיכום ומסקנות:.....39
ביבליוגרפיה:.....40

# הצעת פרויקט משובצות

## פרטי מגיש ההצעה:

סמל מוסד: 595223
שם מכללה: סמינר "גברא" בית יעקב
שם סטודנט: תמר מאזוז
ת"ז: 327821054
שם הפרויקט: מערכת מדידת חלל אוטונומית.

## תקציר הפרויקט:

### רקע המערכת:

בעולם האדריכלות, מדידת חללים מדויקת היא התשתית לכל הפרויקט האדריכלי.
ללא נתונים מדויקים ואמינים בשלבי הייסוד, כל תהליך הביצוע או העיצוב עלול להיתקל בקשיים ובסטיות יקרות.

כיום, מדידה ידנית דורשת בדרך כלל שני אנשים, וקיים קושי במדידת חדרים ארוכים מאוד או במדידת גובה של תקרות גבוהות.
ובנוסף קיים טעויות בזמן העברת הנתונים, מהמטר לדף או מהדף למחשב.
למרות שקיימים מדי-לייזר ידניים, הם עדיין דורשים מהמודד לעמוד בכל פינה ולמדוד ידנית.

פרויקט זה מציע פיתוח רובוט שיבצע את הפעולה הזו ללא הצורך של מעבר לאורך כל הקירות בצורה אוטונומית.

### מטרות המערכת:

- ביצוע מדידה מלאה, ללא צורך מגע יד אדם מרגע ההפעלה.
- שימוש בטכנולוגיית לייזר למדידה מדויקת של הגובה.
- חישוב וניתוח נתונים לבד, כולל שטח ונפח.
- שמירת הנתונים בצורה דיגיטלית, כדי למנוע טעויות.

### קהל יעד:

מערכת זו מיועדת:

- אדריכלים ומעצבי פנים- זקוקים למדידה מדויקת של החללים, לפני תחילת התיכנון.
- מודדים מוסמכים- זקוקים לזה לצורך ייעול תהליך העבודה בשטח, וצמצום הצוותים.
- קבלני שיפוצים- זקוקים לחישוב שטח ונפח מהירים, לצורך הערכת כמויות חומרים.
- חברות נדל"ן- לצורך הכנת תוכניות דירה מדויקת לשיווק.
- חברות איטום- לצורך חישוב מדויק של שטח הגגות או הרצפות הדורשות איטום.

## תיאור הבעיה:

בזמן מדידת חללים רגילה, קיימות מספר בעיות מרכזיות:

- חוסר דיוק בשיטות ידניות- בזמן מדידה לבד במטר ידני- פיזי, המטר יכול לברוח או להתעקם וזה דופק את כל המדידה, ועוד קיימת אפשרות של טעות קטנה בזמן הכתיבה או בזמן העברת הנתונים מהדף למחשב, וזה יכול ליצור טעויות רבות וגרימת נזק רב.
- הקושי בזמן מדידת הגובה- קשה עד בלתי אפשרי למדוד גובה לבד בלי לטפס. זקוקים ל-2 אנשים + סחיבת ציוד כבד כמו סולם ורק כך קיימת אפשרות למדוד, וכל זה בזמן שהשני מחזיק את המטר. אך לפעמיים קיימים חפצים או רהיטים שמפריעים למתיחת הקו הישר מנקודה לנקודה ומוסיפים כמין בטן למטר.
- מורכבות הנדסית- בחדרים בעלי זוויות קטנות או גדולות מ- 90°, כמעט בלתי אפשרי לדייק עם כלים ידניים! ובנוסף לאחר המדידה המודד צריך להתחיל לחשב שטחים ופינות בראש או במחשבון וגם שם קיימות טעויות רבות.
- ההשלכות הכלכליות- בזבוז חומרים, אם חישבו שטח גדול מידי או קטן מידי בגלל טעות במדידה נקנו חומרים מיותרים, או להפך יש חוסר בחומרים מסוימים שצריך לעצור ולעשות שוב הזמנה ולהוסיף עוד. וזה כסף שניזרק! ואפילו לפעמיים קיימים מצבים שהמוצר עצמו אינו שימושי עוד בגלל טעות במדידה, לדוגמא: מודד מדד שטח לצורך הקמת מטבח והחסיר 5 סנטימטר וכשהשיש מגיע הוא לא נכנס במקום שהכינו לו ויש צורך לבזבז עוד כסף כדי להתאים אותו שוב למידות הנכונות.

לכן יש צורך בפיתוח כלי אוטומטי שיחשב הכל בלי טעויות!

## תיאור הפתרון:

הפתרון בפרויקט שלי הוא פיתוח מערכת רובוטית ניידת למדידת חללים.
המערכת תתבסס על שילוב בין יכולת תנועה עצמאית לבין מערך חיישנים מתקדם, וזה מאפשר לרובוט למפות את החדר תוך כדי נסיעה ללא צורך במגע יד אדם. בשונה מהמדידה הידנית המקובלת, הרובוט יכול לבצע בו זמנית מדידות אופקיות (אורך ורוחב) ואנכיות (גובה) לעבד את הנתונים בזמן אמת ולהפיק תוצאות מדויקות של שטח ונפח שמתועדות באופן דיגיטלי.

מבנה המערכת של הרובוט:

- בקר ESP32
- מנועי DC- המנועים שמאפשרים לרובוט לזוז לכל הכיוונים.
- דרייבר מנועים- רכיב שמקשר בין הבקר למנועים והוא נותן למנועים את הכח מהסוללה כדי להסתובב.
- חיישני אינפרה אדום (IR)- חיישן שמודד את המרחק מהקיר, כדי שהרובוט יסע בקו ישר ולא יתנגש בשום דבר.
- חיישן לייזר (VL53L1X) ToF- חיישן גובה שמודד את המרחק לתקרה בדיוק של מילימטרים.

- אנקודרים (Encoders)- רכיב שסופר כמה פעמיים המנוע הסתובב וכך הוא מודד את אורך הקיר.
- חיישן IMU- קיים בתוכו 2 מכשירים:

1. מד תאוצה (Accelerometer)- בודק את התאוצה של הרובוט, וכך הוא בודק את שיפוע הריצפה או את תזוזת הרובוט.
2. Gyroscope- בודק את מהירות הזווית שהרובוט מסתובב, וזה מאפשר לרובוט לבצע פניות של 90° מדויקות, וכן אם זווית החדר היא עקומה והיא פחות או יותר מ 90° הוא ידווח על זה לבקר.

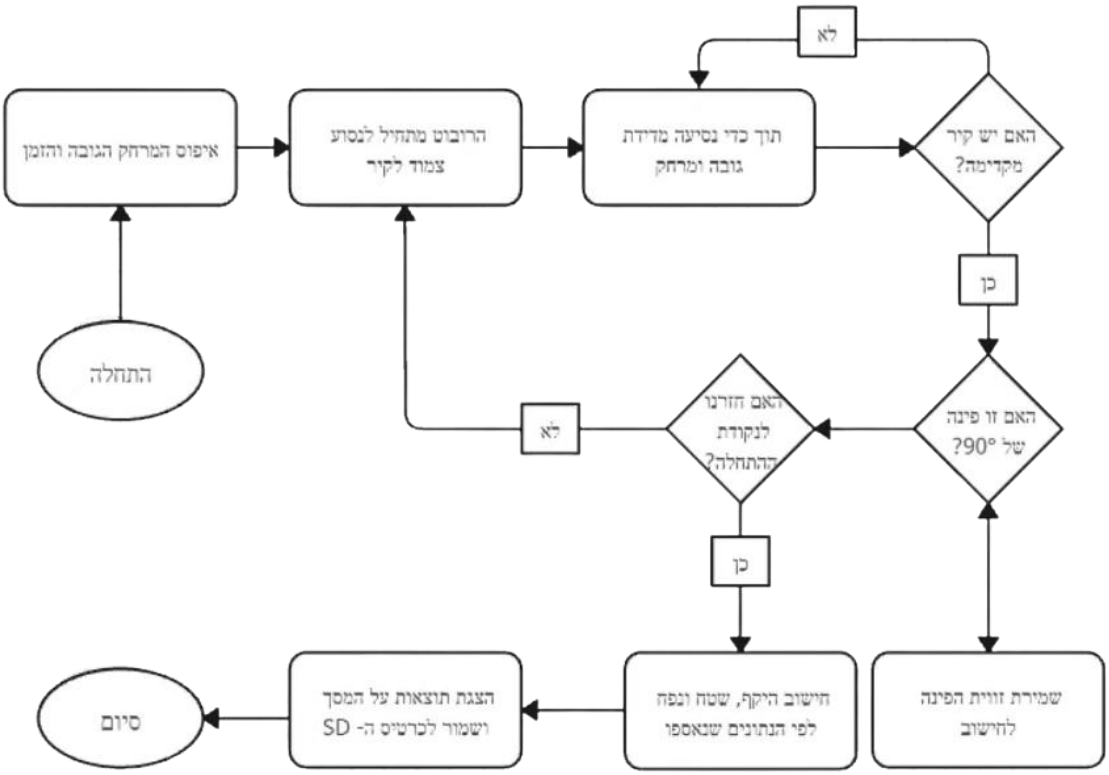
חיישן IMU יחד עם אנקודרים יוצרים ביחד מפה של החדר בתוך הבקר.

- מסך OLED בגודל 64X128.
- כרטיס זיכרון- לשמירת המדידות ואפשרות להעברה אח"כ למחשב.
- זמזם (Buzzer)- מצפצף אם המדידה הנגמרת או כשיש תקלה.
- שעון זמן אמת (RTC)- מוסיף תאריך ושעה לכל מדידה.

## אלגוריתמים המערכת:

- שלב האיפוס (Initialization)- עם הפעלת המערכת, הבקר (ESP32) מאתחל את כל החיישנים, בודק את תקינות כרטיס ה -SD ומאפס את שעון ה-RTC. המשתמש מניח את הרובוט בפינת החדר ולוחץ על כפתור הסטארט.
- נסיעה בשיטת "עוקב קיר (Wall Following)" - הרובוט מתחיל בנסיעה קדימה. חיישני ה- IR מודדים את המרחק מהקיר הצמוד ומבצעים תיקוני היגוי קטנים כדי לשמור על מרחק קבוע ונסיעה בקו ישר לגמרי.
- איסוף נתונים בזמן אמת- תוך כדי נסיעה, האנקודרים סופרים את סיבובי הגלגלים (מדידת אורך הקיר) וחיישן ה- ToF דוגם את גובה התקרה באופן רציף.
- זיהוי וטיפול בפינות- כאשר חיישן ה- IR הקדמי מזהה מכשול (קיר ניצב), הרובוט עוצר. הוא משתמש ב- IMU כדי לבצע פנייה מדויקת. אם הזווית אינה 90°, הבקר שומר את הזווית האמיתית לצורך חישוב השטח.
- סגירת מסלול (Loop Closure)- הרובוט מזהה שחזר לנקודת ההתחלה על ידי השוואת הקואורדינטות בזיכרון. בשלב זה המנועים עוצרים והמערכת מבצעת חישוב סופי של ההיקף, השטח והנפח.
- סיום ותיעוד- התוצאות מוצגות על מסך ה- OLED נשמרות בכרטיס ה- SD עם חותמת זמן מה-RTC וה-Buzzer משמיע צפצוף סיום.

## תרשים זרימה:




חתימת הסטודנט: תמר מאזוז

חתימת המנחה: אביה גברא

תאריך: 20/12/2025

# תקציר:

## רקע לפרויקט:

למדתי אדריכלות בתיכון, ושם למדתי שהפער הכי גדול בתכנון הוא בין מה שרואים על הנייר לבין מה שקורה בפועל בשטח. כמות הנזקים נגרמת לחברות בגלל מדידות לא מדויקות. ולכן החלטתי שהגיע הזמן להפסיק להסתמך על מטר ידני ועל מדידות אנושיות שמביאות לטעויות, ולבנות מערכת שתעשה את העבודה הזו במקומנו.

בתהליכי העבודה המקובלים כיום, מדידת חללים מתבצעת לרוב בשיטות ידניות, המערבות שימוש באמצעים פיזיים ותלויה בכוח אדם. תהליכים אלו אינם רק איטיים ומורכבים, אלא חושפים תוצאות של טעויות בלתי נמנעות, החל מטעויות בזמן המדידה עצמה, ועד לכשלים בהעברת הנתונים לכדי תוכניות עבודה דיגיטליות.

לכל טעות אפילו הכי קטנה בשלב המדידה יש השלכות מצטברות שמתגלות לפני או אפילו רק בשלבי הביצוע. כתוצאה מכך, אנו עדים לא פעם לבזבוז מיותר של חומרים יקרים, חריגות מלוחות זמנים, וצורך בתיקונים יקרים שניתן היה למנוע מראש לו היו מודדים טוב. בנוסף, ישנה מורכבות בזמן מדידת חללים בעלי זוויות לא סטנדרטיות או גבהים משתנים ומורכבות זו מציבה אתגר משמעותיים בפני השיטות הידניות הקיימות. הפרויקט הזה נועד לתת מענה טכנולוגי מתקדם לאתגרים אלו, תוך החלפת המדידה הידנית במערכת אוטונומית המבטיחה דיוק מרבי, חיסכון בזמן ובמשאבי אנוש, ותיעוד אמין המייעל את תהליך התכנון כולו.

## מטרת המערכת:

פיתחתי רובוט אוטונומי שנוסע לאורך הקירות וממפה חללים פנימיים. המטרה היא פשוטה: לקבל נתונים מדויקים (ברמת המילימטר) בלי שבן אדם יצטרך להתרוצץ עם סרט מדידה ולחשב שטחים ונפחים בעצמו. המערכת עושה את כל החישובים לבד ושומרת את המידע בצורה דיגיטלית מסודרת, ככה שאין סיכוי לטעות ברישום או להפסיד חומרים יקרים בגלל חישוב לא נכון.

המטרה המרכזית של המערכת היא ביצוע מדידה מלאה ומדויקת של חללים באופן אוטונומי, תוך יצירת סטנדרט חדש המייתר את הצורך במגע יד אדם מרגע ההפעלה הראשונית. על ידי שימוש בטכנולוגיית לייזר וחיישנים מתקדמים, המערכת שואפת להגיע לדיוק ברמת מילימטרים, ובכך לפתור את הבעיות בחוסר הדיוק הקיימים בשימוש בכלים ידניים מסורתיים .

מעבר לדיוק הטכני, המערכת נועדה לתת פתרון הנדסי משלים לחישובים המורכבים הכרוכים בתכנון חללים. המערכת תבצע באופן עצמאי את ניתוח הנתונים ואת חישובי השטחים הגבהים והנפחים, וזה מאפשר למשתמש לקבל תמונה הנדסית מלאה בזמן אמת, ללא צורך בעבודת חישוב ידנית שעלולה להוות פתח לטעויות .

יתרה מכך, המערכת שמה דגש מיוחד על אמינות הנתונים ושימורם לאורך זמן. על ידי העברת הנתונים באופן דיגיטלי ושמירתם עם חותמת זמן, המערכת מונעת כשלים אנושיים שעלולים להתרחש בשלבי העברת המידע מהשטח למחשב. באופן זה, המערכת לא רק משפרת את איכות המדידה, אלא מהווה כלי עבודה המייעל משמעותית את לוחות הזמנים ומבטיח את איכות התכנון לכל אורך הדרך.

## תיאור הפרויקט:

הפרויקט הזה מציג פיתוח של מערכת רובוטית חכמה וניידת, המיועדת לביצוע מדידות אדריכליות של חללים פנימיים בצורה אוטונומית. המערכת מבוססת על שילוב ייחודי בין יכולת תנועה עצמאית

לבין מערך חיישנים מתקדם, המאפשרים לרובוט לבצע מיפוי של החדר בזמן אמת, ללא תלות במגע יד אדם.

השתמשתי בבקר ESP32 בתור ה'מוח' של המערכת, הוא אחראי על כל איסוף הנתונים מהחיישנים ועיבודם בזמן אמת. הרובוט מצויד במערך חיישנים הכולל חיישן לייזר למדידת גובה מדויקת ברמת מילימטרים, חיישני אינפרה-אדום (IR) המבטיחים נסיעה בקו ישר ומניעת התנגשויות, וחיישן IMU המשלב מד תאוצה וג'ירוסקופ לצורך ניווט מדויק, ביצוע פניות וזיהוי זוויות. בנוסף, נעשה שימוש באנקודרים המותקנים על המנועים כדי למדוד במדויק את המרחקים שעובר הרובוט לאורך הקירות.

תהליך העבודה של המערכת בנוי בצורה מובנית: עם הפעלתה, הרובוט מבצע הליך איפוס ומתחיל בנסיעה לאורך קירות החדר בשיטת "עוקב קיר" (Wall Following) תוך כדי תנועה, המערכת בודקת באופן רציף את המרחקים והגבהים, ומזהה פינות באופן אוטומטי באמצעות הבקר. עם השלמת הקפת החדר וחזרה לנקודת ההתחלה (סגירת מעגל), המערכת מבצעת חישוב סופי של היקף, שטח ונפח החלל. כל הנתונים שמורים באופן דיגיטלי על גבי כרטיס זיכרון, עם חותמת זמן מדויקת המופקת משעון זמן אמת (RTC), ומוצגים על גבי מסך OLED המותקן על גבי הרובוט.

## קהל הייעד:

הפרויקט הזה מיועד לכל מי שצריך לעבוד עם חללים ורוצה לעשות את זה מהר ובלי טעויות:

- אדריכלים ומעצבי פנים- עבורם, דיוק בנתוני החלל הוא הכי חשוב לתכנון מוצלח. המערכת מאפשרת להם לקבל תשתית נתונים אמינה ומדויקת כבר בשלבים הראשונים של הפרויקט, מה שחוסך זמן יקר של מדידות ידניות ומבטיח שהתוכניות תואמות את המציאות בשטח.
- קבלני שיפוצים איטום ובינוי- בעלי מקצוע אלו זקוקים לחישובים מהירים ומדויקים של שטחים ונפחים לצורך הערכת כמויות וחומרי גלם. השימוש במערכת הזו מאפשרת להם לקבל נתונים מיידיים מהשטח, ובכך לייעל את הצעות המחיר ולמנוע חריגות או בזבוז מיותר של חומרים.
- חברות נדל"ן וניהול נכסים- חברות אלו נדרשות לתיעוד דיגיטלי מהיר של נכסים לצורכי שיווק או תחזוקה. היכולת להפיק נתונים מדויקים בצורה אוטונומית מהווה עבורן כלי עבודה משמעותי לשיפור ניהול המידע הנדל"ני.

באופן כללי, הפרויקט פונה לכל גורם הנדרש להפיכת חלל פיזי לתוכנית עבודה מפורטת, ומחפש דרך לשלב חדשנות טכנולוגית המבטיחה אמינות, מהירות וחיסכון בחומרים.

# תצהיר חתום:

**אני מצהירה שאני כתבתי את הפרויקט הזה ללא העתקות.** תמר

# מושגים:

- עוקב קיר (Wall Following)- האלגוריתם המאפשר נסיעה ישרה במקביל לקיר, באמצעות בדיקה בכל רגע נתון מה המרחק מהקיר.
- סגירת מסלול (Loop Closure)- הזיהוי שהרובוט חזר לנקודת ההתחלה.
- שעון זמן אמת (RTC)- תיעוד המדידות עם חותמת זמן.
- דרייבר מנועים- הרכיב המתווך בין הבקר למנועים.
- כיול חיישנים (Sensor Calibration)- התהליך שבו המערכת לומדת את רמת הדיוק של החיישנים כדי להפחית סטיות מדידה ולהבטיח שהנתונים הנקלטים יהיו מדויקים לתוצאות במציאות.
- פסיקת מערכת (Interrupts)- מנגנון בבקר שמאפשר למערכת להגיב באופן מיידי לאירועים חיצוניים (כמו זיהוי קרבה או פעימת אנקודר) תוך כדי תנועה, מה שחייב לדיוק המדידה.
- מודול SD-Card- הרכיב המאפשר את שמירת המידע בצורה לא נדיפה, כך שהנתונים שנאספו בשטח יישמרו גם לאחר ניתוק מתח מהמערכת.
- תקשורת I2C- הפרוטוקול שדרכו הבקר מתקשר עם רכיבים שונים (כמו חיישן הלייזר וה-OLED) כדי להעביר את הנתונים בצורה אמינה ומהירה.
- אלגוריתם דגימה (Sampling Rate)- הגדרת תדירות הבדיקה של החיישנים במהלך הנסיעה, שקובעת את רזולוציית המיפוי של החדר (כמה נקודות מדידה אנחנו אוספים לאורך כל מטר של קיר).
- קואורדינטות מקומיות (Local Coordinate System)- מערכת הצירים שהרובוט מגדיר לעצמו כדי לתרגם את המרחקים והזוויות שהוא מודד לנתונים גיאומטריים של שטח ונפח.
- בקרת מנועים (PWM - Pulse Width Modulation)- השיטה שבה המערכת שולטת על מהירות ודיוק התנועה של המנועים, מה שמאפשר נסיעה יציבה וחלקה לאורך הקירות.

# תיאור הנושא:

לאחר שראיתי כמה תהליך המדידה הידנית הוא מיושן, קשה ומועד לטעויות, יצאתי לפתח רובוט שיעשה את העבודה הקשה עבורנו בדיוק מושלם ובמינימום מאמץ.
בניתי רובוט שמבצע סריקה אוטונומית ללא מגע יד אדם ובכך חוסך לנו טעויות מרובות בעת המדידה או בזמן העתקת נתונים.

המערכת קלה מאוד לתפעול.
המשתמש בסך הכל צריך להניח את הרובוט צמוד לאחד הקירות ולוחץ על כפתור ההפעלה באתר או על הרובוט עצמו. לאחר מכן אין צורך לגעת יותר ברובוט, הרובוט מתחיל בנסיעה והוא תמיד ניצמד לצד ימין (באמצעות אלגוריתמים "עוקב קיר") ועל ידי כך הוא יכול להשלים את היקף החדר בצורה בטוחה כך שהוא לא מפספס שום קיר או חלל צמוד.
לסיום, כשהרובוט מגיע לנקודת ההתחלה ע"י השוואת קואורדינטות (X,Y) הוא נעצר באופן אוטומטי, מבצע חישוב גיאומטרי של השטחים והנפחים ומציג את הנתונים על גבי המסך, הקיים על גב הרובוט וגם שולח את כל הנתונים כולל שרטוט החדר בצורה פשוטה לאתר כקובץ הורדה.

איך הרובוט מבצע את כל המדידות בצורה טובה ומדויקת, ולא נתקע בקיר או שאינו נעצר לפני סיום המדידה?

היכולת של המערכת לפעול בצורה מדויקת וחלקה מתבססת בעיקר על מערך חיישנים, המחוברים לבקר המרכזי ומתפקדים כ'מוח' של הרובוט. כדי למדוד את אורכי הקירות במדויק, מותקנים אנקודרים על המנועים, אשר סופרים את סיבובי הגלגלים ומתרגמים אותם למידות אורך מדויקות. לצדי הרובוט ממוקמים ארבעה חיישני קרבה, שסורקים את הסביבה בכל רגע נתון ומבטיחים שהרובוט ישמור על מרחק עבודה קבוע של 2 ס"מ מהקיר, ללא סטייה. בראש המערכת מותקן חיישן לייזר הסורק את גובה החלל באופן רציף כדי לזהות מכשולים ולמנוע התנגשויות, ובנוסף, הוספתי באזר שמתריע קולית על סיום תהליך המדידה או על חריגות בדרך, מה שמעניק למשתמש אינדיקציה ברורה גם כשהוא לא מסתכל על המסך. כל הנתונים, הכוללים את המידות והשרטוט המדויק, נשלחים בסיום הפעולה לאתר האינטרנט שפיתחתי, שם הם מרוכזים כקובץ מסודר להורדה, מה שחוסך את הצורך בהעתקה ידנית ומבטיח שהמידע יהיה זמין ונגיש לעבודה מיידית.

# תיאור הבעיה:

היום, כשאדריכלים או קבלנים צריכים למדוד חלל, הם עדיין עובדים בשיטות של פעם. לוקחים סרט מדידה, מתרוצצים בחדר, רושמים מספרים על דף – וזה תהליך שפשוט מוביל לטעויות. טעות קטנה ברישום של סנטימטר אחד כאן או שם, הופכת מהר מאוד לבעיה גדולה בשלב הביצוע הפרויקט, כשמזמינים חומרים או כשמנסים להתאים רהיטים וקירות למקום.

חוץ מהדיוק, יש כאן גם עניין של זמן ומאמץ. למדוד חללים זה תהליך איטי, שלפעמים דורש שני אנשים, וזה פשוט לא מסתדר עם הקצב של עולם התכנון המודרני, שבו הכל היום דיגיטלי וממוחשב. אני חושבת שחסר בשוק כלי שיהיה "חכם" מספיק כדי לקחת את העבודה הזו מהאדם, ולעשות אותה בצורה שקטה, אוטונומית ומדויקת, בלי שנצטרך לחשוש ששכחנו למדוד פינה או שרשמנו מספר לא נכון.

אז מכאן קם הפרויקט שלי ומטרתו לאפשר לכל איש מקצוע להניח מכשיר בחדר, ללחוץ על כפתור, ולקבל תוך דקות ספורות את המידות המדויקות, כולל שטח ונפח, בלי שום עבודת הכנה מורכבת. המטרה הסופית היא לחסוך זמן יקר, למנוע טעויות יקרות ולתת למתכננים שקט נפשי שמה שהם רואים בתוכנית במחשב – זה בדיוק מה שיש בשטח.

# ניתוח חלופות מערכתי:

בשלב התכנון של המערכת, לא רצתי ישר לקנות רכיבים. עשיתי בדיקה של כמה אפשרויות כדי לוודא שאני בוחרת את הכלי שהכי מתאים למשימה שלי, שיהיה גם מדויק, גם אמין וגם יעבוד טוב עם הפיצ'רים שרציתי (כמו שליחת נתונים לאתר).

## בחירת הבקר המרכזי (ESP32 לעומת Arduino)

ההתלבטות העיקרית שלי הייתה בין ה-Arduino המוכר לבין ה-ESP32. הבנתי מהר מאוד שהפרויקט שלי דורש יותר מרק הפעלת מנועים – הוא צריך להתחבר לרשת, לעבד נתונים ולתקשר עם האתר.

| | **Arduino Uno** | **ESP32** |
|---|---|---|
| מהירות מעבד | נמוכה (16MHz) | גבוהה מאוד (240MHz) |
| קישוריות | אין (דורש מודול חיצוני) | Wi-Fi ו-Bluetooth מובנים |
| זיכרון | קטן מאוד | גדול (מספיק לעיבוד נתונים) |
| מתאים לפרויקט | לא | כן |

הבחירה ב-ESP32 הייתה ברורה. בגלל שהמערכת שלי שולחת שרטוטים לאתר, הייתי חייבת בקר עם Wi-Fi מובנה. להשתמש ב-Arduino היה מסבך את כל המערכת עם מודולים חיצוניים שתופסים מקום ומקטינים את האמינות. ה-ESP32 נתן לי את כל מה שאני צריכה בלוח אחד, קטן וחזק.

## חיישן מדידת מרחק (לייזר ToF לעומת אולטרסאונד)

כדי למדוד את החלל, הייתי צריכה חיישן שיכול לתת לי מידות מדויקות של הקירות. בדקתי את חיישן האולטרסוני הפשוט מול חיישן לייזר (ToF).

| | **אולטרסוני (HC-SR04)** | **לייזר (ToF - VL53L0X)** |
|---|---|---|
| דיוק | בינוני (רגיש לרעשים) | גבוה (ברמת מילימטר) |
| זווית סריקה | רחבה (לא מדויק לפינות) | צרה (קרן ממוקדת) |
| אמינות | מושפע מהחזרים מהקיר | יציב וקבוע |
| מתאים לפרויקט | לא | כן |

חיישן האולטרסוני אולי זול יותר, אבל הוא לא מספיק מדויק בשביל מדידות של חדרים. אם המטרה שלי היא דיוק של מילימטרים כדי שלא ייווצר בזבוז חומרים, חיישן הלייזר הוא הפתרון היחיד שנותן לי ראש שקט. הוא ממוקד, עקבי, ולא משתגע כשהוא מגיע לפינה של החדר.

## בחירת שיטת תקשורת (Wi-Fi לעומת כרטיס SD)

היה לי ברור שהמערכת צריכה לשמור את המידע, אבל התלבטתי איך להוציא אותו מהרובוט למשתמש. אפשרות אחת הייתה פשוט להשתמש בכרטיס SD, ואפשרות שנייה הייתה שידור ישיר לאתר אינטרנט.

| | **כרטיס זיכרון (SD)** | **תקשורת Wi-Fi לאתר** |
|---|---|---|
| נוחות למשתמש | נמוכה (צריך להוציא פיזית) | גבוהה (אוטומטי) |
| זמינות נתונים | לא מיידית | בזמן אמת |

| חוויית שימוש | מיושנת | מודרנית |
|---|---|---|
| מתאים לפרויקט | לא | כן |

בחרתי ב-Wi-Fi כי המערכת שלי נועדה לייעל את העבודה. ברגע שהרובוט מסיים למדוד, המשתמש מקבל את כל הנתונים ישירות לאתר בלי להסתבך עם כרטיסים, קוראים או מחשבים. זה הופך את המערכת לכלי עבודה מקצועי שמרגיש כמו "מוצר מדף" מתקדם.

## אלגוריתם נסיעה (עוקב קיר לעומת SLAM מורכב)

כאן הייתה הדילמה הכי גדולה: איך הרובוט יסרוק את החדר? יכולתי ללכת על אלגוריתמים מורכבים של מיפוי (כמו SLAM), אבל הבנתי שזה ידרוש כוח חישוב שהרובוט לא צריך.

| | **SLAM (מיפוי מתקדם)** | **עוקב קיר (Wall Following)** |
|---|---|---|
| מורכבות | גבוהה מאוד | פשוטה ויעילה |
| משאבי חומרה | דורש מחשב חיצוני | פועל על בקר פשוט |
| אמינות בשטח | תלוי בתנאי תאורה/רעש | יציב מאוד בחדרים |
| מתאים לפרויקט | לא | כן |

האלגוריתם של "עוקב קיר" מושלם למשימה שלי. המטרה היא למדוד חללים סגורים, והנסיעה לאורך הקירות היא הדרך הכי מדויקת להקיף כל פינה בלי שהרובוט "ילך לאיבוד". זה פתרון אלגנטי, מהיר ולא כבד על המעבד, מה שמאפשר לי לשמור על דיוק גבוה בלי להסתבך עם אלגוריתמים מיותרים.

## ניהול הנעת הרובוט (מנועי DC עם דרייבר לעומת מנועי סרוו)

הייתי צריכה להחליט איך הרובוט יזוז. מצד אחד, סרוו זה קל לתכנות, אבל מצד שני, מנועי DC עם דרייבר מאפשרים שליטה אמיתית במהירות ובכוח.

| | **מנועי סרוו (Servo)** | **מנועי DC עם דרייבר (L298N/TB6612)** |
|---|---|---|
| יכולת נסיעה | מוגבלת (סיבוב חלקי) | חופשית ורציפה |
| דיוק בנסיעה | נמוך בנסיעות ארוכות | גבוה (בשילוב אנקודרים) |
| שליטה במהירות | קשה | מדויקת (PWM) |
| מתאים לפרויקט | לא | כן |

כדי למפות חדרים, הרובוט חייב לנסוע בצורה רציפה וחלקה לאורך הקיר. סרוו לא נועד לנסיעה רציפה, לעומת זאת, מנועי DC שמאפשרים שליטה במהירות באמצעות PWM (Pulse Width Modulation) נתנו לי את הגמישות לכוון את מהירות הרובוט כך שהחיישנים יספיקו לקרוא את הנתונים בצורה מדויקת מבלי להחמיץ אף פינה. השילוב שלהם עם אנקודרים הפך את הנסיעה למדויקת וניתנת לחישוב מתמטי.

# תיאור החלופה הנבחרת:

אחרי שבדקתי כמה אפשרויות שונות והשוואתי אותן למה שהפרויקט שלי באמת צריך, בחרתי בפתרון המערכתי שמשלב ביצועים טובים, אמינות ותפעול פשוט. בבסיס של המערכת הזו יושב בקר ה -ESP32, שמתפקד בתור המוח של הרובוט, ואליו חיברתי חיישני לייזר ומערכת אנקודרים שאחראים על הניווט והמדידה המדויקת.

הבחירה שלי בדרך הזו הגיעה כי רציתי פתרון אחד שנותן מענה להכל. השילוב בין המנועים עם האנקודרים לבין חיישני הלייזר נותן לי תוצאות ברמת המילימטר, מה שסוף סוף פותר את בעיית חוסר הדיוק שקיימת היום בכל כלי המדידה הישנים. מעבר לזה, בזכות אלגוריתם ה"עוקב קיר" שבניתי, הרובוט מסתדר לבד לגמרי – הוא סורק חללים מורכבים בלי שאני צריכה לגעת בו או להגדיר לו הגדרות מחדש בכל פעם.

מה שהכי אהבתי בפתרון הזה הוא עניין החיבור לרשת. בגלל שהבקר שלי כולל Wi-Fi מובנה, המידע עובר מהרובוט ישירות לאתר האינטרנט שבניתי בזמן אמת. זה חוסך למשתמש את כל כאב הראש של העברת קבצים או התעסקות עם חישובים, כי הכל מגיע מוכן להורדה ישר למחשב או לטלפון. בנוסף, הוספתי רכיב זיכרון חיצוני (כרטיס SD) המשמש כגיבוי מקומי לנתונים. כך, גם במקומות ללא קליטת Wi-Fi, המערכת ממשיכה לתעד כל מדידה בבטחה ולמנוע אובדן מידע קריטי. לסיכום, השילוב שבניתי כאן מחבר בין חומרה איכותית לתוכנה חכמה, ובסוף זה נותן לאדריכלים ולשיפוצניקים תמונה הנדסית מלאה של החלל בלי שהם יצטרכו להתאמץ או לחשוש מטעויות.

# הסבר על הבקר:

## הסבר על מיקרו-בקר ולוח פיתוח ESP32-DevKitC (38-Pin, USB Type-C)

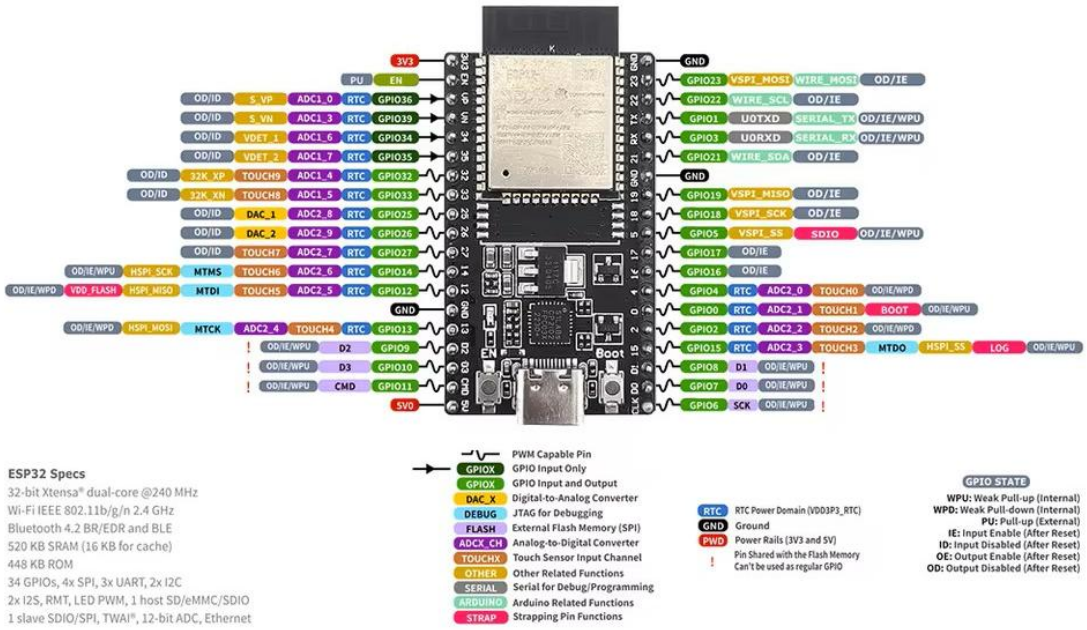
רכיב זה מהווה את יחידת העיבוד והבקרה המרכזית (ה"מוח") של המערכת האוטונומית, והוא מופעל באמצעות מתח הזנה המיוצב על גבי הלוח.

ה-ESP32 הוא מיקרו-בקר עוצמתי בעל ארכיטקטורת ליבה כפולה (Dual-Core) בטכנולוגיית 32 סיביות, המאפשר לו לבצע עיבוד מקבילי ולנהל מספר משימות בו-זמנית. שבב זה פועל בתדר שעון דינמי של עד MHz240 וכולל זיכרון RAM פנימי בנפח של 512 KB, המקנה לו את כוח המחשוב הדרוש לניהול מערך החיישנים, קריאת האנקודרים וביצוע חישובי שטח ונפח מורכבים בזמן אמת. הלוח כולל קישוריות אלחוטית מובנית של Wi-Fi ו-Bluetooth (בתקן BLE), רכיבי הצפנה מואצת בחומרה ומנגנוני אבטחה מתקדמים, מה שמאפשר הרחבה עתידית של המערכת ליישומי ה-IoT ושליחה אלחוטית של הנתונים.

לוח הפיתוח הספציפי שבחרתי לפרויקט מבוסס על תצורת ESP32-DevKitC המשופרת הכוללת 38 פינים (Pin-38) וחיבור USB Type-C מודרני, המשמש להעלאת הקוד ולניפוי באגים (Debugging) דרך גשר תקשורת מובנה מסוג CP2102 (USB to UART). הלוח תומך בטווח רחב של ממשקי תקשורת היקפיים כגון I2C (עבור שעון ה-RTC, מסך ה-OLED וחיישן ה-ToF) , פיני ADC לקריאה אנלוגית, ומחוללי אותות PWM (לשליטה על מהירות מנועי הנסיעה דרך הדרייבר). הלוח מיוצב לעבודה חסכונית באנרגיה וכולל וסת מתח פנימי המאפשר להזין אותו דרך חיבור ה-USB או באמצעות ספק כוח וסוללה חיצונית.

לצורך ייעול החיווט והגברת היציבות המכנית של הרובוט, לוח הפיתוח משולב עם ערכת לוח הרחבה GPIO. לוח הרחבה זה מפצל כל פין של הבקר לשלשות ייעודיות של פינים (מתח, אדמה ואות - VCC, GND, Signal), מה שמאפשר חיבור ישיר, מהיר ובטוח של כל רכיבי הקצה והחיישנים במערכת ללא צורך בהלחמות מורכבות או בשימוש במטריצות חיווט רגישות. פיתוח הקוד ותכנות הבקר מתבצעים בצורה מלאה בסביבת העבודה הרשמית של Arduino IDE בשפת C/C++.






# Functional Description

| Key Component | Description |
|---|---|
| ESP32-WROOM-32 | A module with ESP32 at its core. |
| EN | Reset button. |
| Boot | Download button. Holding down Boot and then pressing EN initiates Firmware Download mode for downloading firmware through the serial port. |
| USB-to-UART Bridge | Single USB-UART bridge chip provides transfer rates of up to 3 Mbps. |
| Micro USB Port | USB interface. Power supply for the board as well as the communication interface between a computer and the ESP32-WROOM32 module. |
| 5V Power On LED | Turns on when the USB or an external 5V power supply is connected to the board. |
| I/O | Most of the pins on the ESP module are broken out to the pin headers on the board. You can program ESP32 to enable multiple functions such as PWM, ADC, DAC, I2C, I2S, SPI, etc. |

# הסבר על חיישנים:

## חיישן טווח לייזר ToF (VL53L1X)




חיישן זה מופעל באמצעות פרוטוקול תקשורת דיגיטלי מסוג I2C ומיועד למדידת מרחקים מדויקת במערכת.

בתוך החיישן קיים מנגנון דיאודת לייזר VCSEL בלתי נראית באורך גל של 940nm, המשגרת פולסי אור ומודדת בדיוק רב את זמן חזרתם (טכנולוגיית Time of Flight - ToF) מהתקרה כדי לחשב את גובה החלל במדויק. מדובר בחיישן טווח מתקדם שפועל במתח הזנה (VCC) של 3.3 וולט או 5 וולט (הודות למייצב מתח מובנה על המודול), כאשר צריכת הזרם שלו נמוכה מאוד ונעה סביב כ-20mA בלבד בזמן פעולה רגילה.

טווח המדידה של החיישן מתאים למרחקים של עד 4 מטרים, והוא מסוגל לספק נתוני גובה מדויקים ברמת המילימטר ללא תלות בצבע התקרה, בחומר ממנו היא עשויה או בתנאי התאורה והצללים בחדר. למודול קיימים לרוב פיני מתח (VCC ו-GND), פיני תקשורת (SDA ו-SCL) ופין בקרה (XSHUT) המאפשר כיבוי זמני או שינוי כתובת. יש אפשרות לשלוט בחיישן ולקרוא ממנו נתונים רציפים בזמן נסיעת הרובוט ישירות דרך בקר ה-ESP32 באמצעות ספריות קוד ייעודיות, כיוון שהבקר מסוגל לספק את זרם העבודה הנמוך הנדרש להפעלתו ללא צורך בדוחף זרם חיצוני.

## רכיב התרעה קולית(Buzzer)



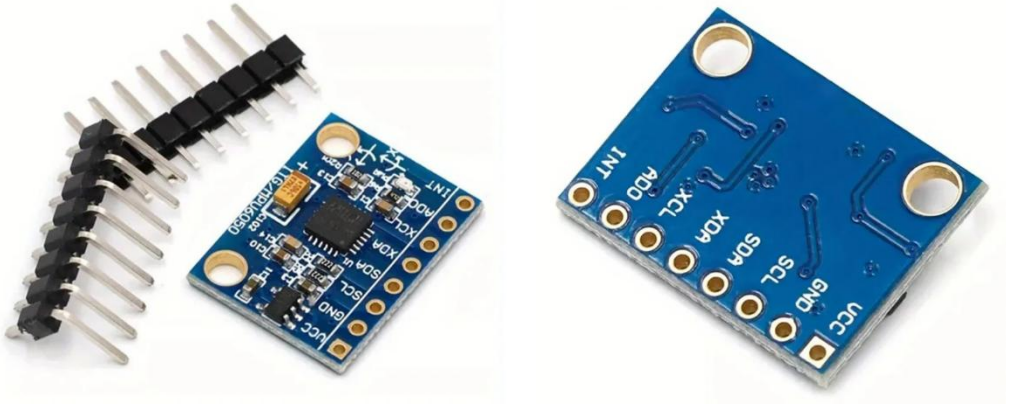

רכיב זה מופעל באמצעות מתח ישר (DC) ומיועד להפקת חיווים קוליים, התרעות על תקלות ואיתות על סיום תהליך המדידה של החלל .

בתוך הבאזר קיים מנגנון אלקטרומגנטי או פייזואלקטרי הגורם לתזוזה ורטט מהירים של דיסק מתכתי דק, ובכך נוצר הצליל האקוסטי. מדובר ברכיב הפועל במתח נמוך של 3 וולט (V3), המתאים לעבודה ישירה מול פיני בקר ה-ESP32 הפועלים ברמות מתח של V3.3. צריכת הזרם של הבאזר

בזמן פעולה היא נמוכה מאוד (כ-mA10 עד mA30), מה שמאפשר להפעיל אותו ישירות מפין דיגיטלי של הבקר ללא צורך ברכיבי הגנה או בדוחף זרם חיצוני.

לבאזר שני פינים בלבד – פין חיובי (VCC, מסומן לרוב ברגל ארוכה או בסימן +) ופין שלילי (GND). אופן השליטה ברכיב מתבצע ישירות מהקוד: עם סיום מיפוי החדר וסגירת המסלול ( Loop Closure), הבקר מפיק אות לוגי HIGH (3.3V) למשך זמן קצר כדי להשמיע צפצוף ברור למשתמש המודיע על השלמת החישובים ושמירת הנתונים לכרטיס ה-SD.

## רכיב מדידת תנועה MPU6050 (GY-521):

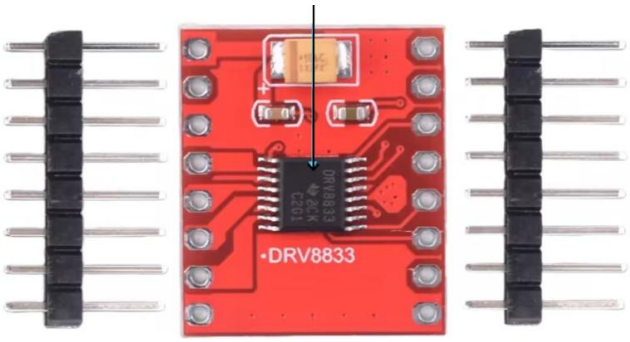



מודול זה מופעל באמצעות פרוטוקול תקשורת דיגיטלי מסוג I2C ומיועד למדידת מיקום, תנועה וזווית במרחב.

בתוך הרכיב קיים מנגנון משולב של מד-תאוצה תלת-צירי (Accelerometer) ומד-סחרור תלת-צירי (Gyroscope), המודדים יחד שינויי מהירות וזווית ב-6 צירים שונים (DOF-6). בנוסף, הרכיב מכיל מעבד תנועה דיגיטלי פנימי (DMP - Digital Motion Processor) המבצע את חישובי הזווית המורכבים ישירות על הצ'יפ כדי להפחית עומס מהבקר הראשי. מדובר ברכיב מתקדם שפועל במתח הזנה (VCC) של בין 3.3 וולט ל-5 וולט (הודות למייצב מתח 3.3V מובנה על הלוח), כאשר צריכת הזרם שלו נמוכה במיוחד ונעה סביב כ- 4mA בלבד בזמן עבודה.

לרכיב קיימים 8 פינים – ביניהם פיני המתח (VCC ו-GND), פיני התקשורת הסטנדרטיים (SDA ו-SCL), ופיני בקרה מתקדמים: פין AD0 (לקביעת כתובת ה-I2C של החיישן), פין INT (להפקת פסיקות לבקר כשיש נתון מוכן), ופיני XDA ו-XCL המשמשים לחיבור רכיבי ניווט חיצוניים נוספים (כמו מצפן דיגיטלי) ישירות לחיישן. יש אפשרות לשלוט ברכיב ולקרוא ממנו נתונים ישירות דרך בקר כמו ESP32 או Arduino באמצעות ספריות קוד ייעודיות (כמו MPU6050 או Adafruit_MPU6050), והוא אינו דורש רכיבי דחיפת זרם חיצוניים.

## דוחף מנועים (Motor Driver / Driver)

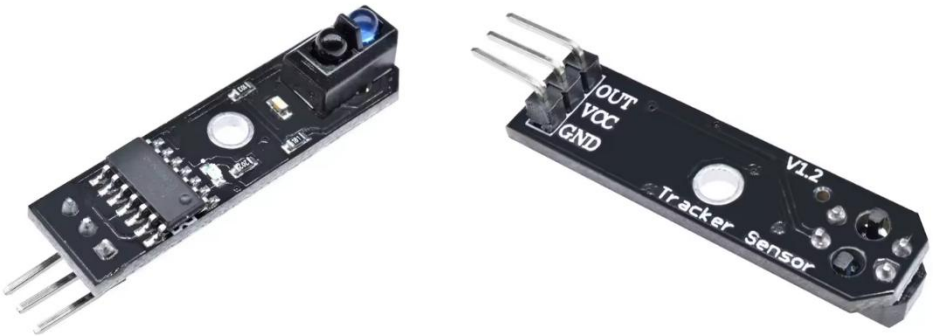



רכיב זה מופעל באמצעות אותות בקרה דיגיטליים מסוג PWM מהבקר ומיועד לשליטה במהירות ובכיוון הנסיעה של מנועי ה-DC המניעים את הרובוט.

בתוך הרכיב קיים מנגנון "גשר H" (H-Bridge) המבוסס על טרנזיסטורים, המאפשר להפוך את כיוון זרימת המתח המגיע מהסוללה אל המנועים, ובכך לשלוט על כיוון התנועה קדימה, אחורה וביצוע פניות מדויקות בפינות החדר. מדובר ברכיב המקשר (דרייבר) שמפריד בין מערכת הפיקוד העדינה למערכת הכוח, בעוד שבקר ה-ESP32 מספק רק את אותות הפיקוד החלשים, הדרייבר מסוגל לעמוד בזרמי ההתנעה הגבוהים של מנועי הדי-סי (שיכולים להגיע למאות מיליאמפרים) ומזרים אליהם את הזרם הנדרש ישירות ממקור המתח החיצוני.

הדרייבר כולל פיני כניסת מתח מהסוללה (VCC ו-GND), פיני כניסת אותות בקרה מהבקר (Inputs), ופיני יציאות כוח המתחברים ישירות אל הדקי המנועים (Outputs). חיבור המערכת דרך רכיב דוחף זרם זה הוא הכרחי, מכיוון שפיני ה-ESP32 עצמם אינם מסוגלים לספק זרם גבוה ועבודה מולם הייתה גורמת לשריפת הבקר.

## חיישן מרחק אינפרה-אדום (IR Tracker Sensor)



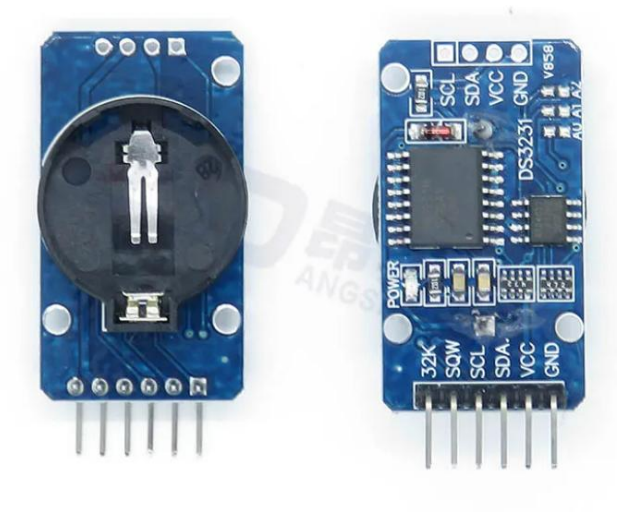

חיישן זה מופעל באמצעות מתח ישר (DC) ומיועד לזיהוי מכשולים, מדידת מרחק יחסי ומעקב קיר במערכת האוטונומית.

בתוך החיישן קיים מנגנון משולב הכולל דיאודת משדר אינפרא-אדום (IR LED) המשגרת אור בתדר בלתי נראה, ודיאודת מקלט (Phototransistor) הרגישה לאור המוחזר מהקיר או מהעצם שמולו. החיישן מודד את עוצמת האור המוחזר ומתרגם אותה לאות חשמלי המעיד על קרבה למכשול. מדובר ברכיב פשוט ואמין שפועל במתח הזנה (VCC) של בין 3.3 וולט ל-5 וולט (המתאים ישירות

לעבודה מול רמות המתח של הבקר), כאשר צריכת הזרם שלו נמוכה מאוד ועומדת על כ-mA10 עד mA15 בלבד בזמן פעולה.

למודול קיימים 3 פינים בלבד – פיני המתח (VCC ו-GND) ופין יציאת אות דיגיטלי (OUT). אופן הפעולה שלו בפרויקט מאפשר לבקר ה-ESP32 לקרוא את מצב הפין הדיגיטלי בצורה רציפה : כאשר הרובוט נוסע ומבצע תיקוני היגוי קטנים כדי לשמור על מרחק קבוע מהקיר, החיישן מספק חיווי מיידי לבקר. אם הרובוט מתקרב מדי לקיר או מזהה קיר ניצב מקדימה, האות בפין OUT משתנה (מצב לוגי HIGH או LOW, בהתאם לכיול), מה שמאפשר לבקר לעצור את המנועים או לבצע פנייה מדויקת, ללא צורך ברכיבי דחיפת זרם חיצוניים.

## רכיב שעון זמן אמת DS3231 (RTC Module)

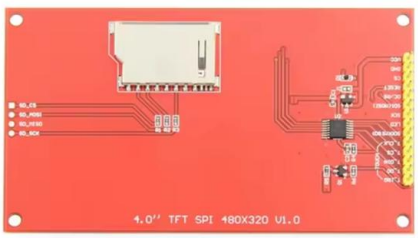



רכיב זה מופעל באמצעות פרוטוקול תקשורת דיגיטלי מסוג I2C ומיועד לשמירה, ניהול ומעקב מדויק אחר הזמן (שעות, דקות, שניות) והתאריך במערכת.

בתוך הרכיב קיים מנגנון מתנד קריסטל פנימי ומובנה, הכולל חיישן טמפרטורה המשמש לפיצוי וכיול אוטומטי של תדר המתנד (TCXO). מנגנון זה מונע סטיות זמן הנובעות משינויי טמפרטורה סביבתיים ומבטיח דיוק גבוה במיוחד לאורך שנים. בנוסף, המודול כולל תושבת לסוללת גיבוי חיצונית (לרוב סוללת כפתור מסוג CR2032), המאפשרת לשעון להמשיך לפעול ולשמור על הזמן הנכון גם כאשר הרובוט כבוי או מנותק לחלוטין ממקור המתח הראשי שלו. מדובר ברכיב מתקדם שפועל במתח הזנה (VCC) של בין 3.3 וולט ל-5.5 וולט, כאשר צריכת הזרם שלו נמוכה ביותר.

למודול קיימים פינים משני צדדיו, כאשר הפינים המרכזיים לחיבור לבקר הם פיני המתח (VCC ו-GND) ופיני התקשורת הסטנדרטיים (SDA ו-SCL). קיימים גם פיני הרחבה נוספים כמו SQW (להפקת פולסים בתדר קבוע או התראות) ו-K32 (ליציאת תדר השעון הגולמי). יש אפשרות לאתחל את הרכיב, לקבוע את הזמן הראשוני ולקרוא ממנו את נתוני השעה והתאריך הנוכחיים ישירות דרך בקר ה-ESP32 באמצעות ספריות קוד ייעודיות (כמו RTClib), והוא אינו דורש רכיבי דחיפת זרם חיצוניים.

# רכיב צג גרפי צבעוני ומודול כרטיס זיכרון (4.0" TFT SPI LCD Display with SD Card Slot)




Bottom



TOP

רכיב זה מופעל באמצעות פרוטוקול תקשורת דיגיטלי מהיר מסוג SPI (Serial Peripheral Interface) ומיועד להצגת נתוני המדידות בזמן אמת למשתמש, וכן לשמירה פיזית של קובצי הנתונים בתוך זיכרון נייד.

המודול משלב שתי פונקציות מרכזיות על גבי כרטיס אלקטרוני אחד:

1. מנגנון התצוגה (TFT LCD): מסך בגודל 4.0 אינץ' המבוסס על טכנולוגיית טרנזיסטורים דקי קרום (Thin-Film Transistor). טכנולוגיה זו מאפשרת שליטה פרטנית ומהירה על כל פיקסל, ומספקת תצוגה צבעונית עשירה ברזולוציה גבוהה של 480 X 320 פיקסלים. רכיב התצוגה מנוהל על ידי שבב דחף (Driver IC) ייעודי המקבל פקודות גרפיות מהבקר ומדליק את נוריות ה-LED האחוריות (Backlight) כדי להציג טקסטים, מדדים וגרפים ברורים של מבנה החדר.
2. מנגנון כרטיס הזיכרון (SD Card Slot): בחלקו האחורי של המודול מובנית תושבת לכרטיס זיכרון מסוג Micro SD. תושבת זו מתממשקת ישירות לקווי תקשורת ה-SPI של הבקר, ומאפשרת לו לפתוח, לכתוב ולשמור קבצים דיגיטליים (כגון קובצי טקסט או CSV) המכילים את ערכי אורך הקירות, גובה התקרה וחישובי השטח.

המודול פועל במתח הזנה (VCC) של 5 וולט או 3.3 וולט (כולל מייצב מתח מובנה על הלוח). בשל כמות הפיקסלים והצורך בהעברת מידע רב ומהיר בצבע, הוא עושה שימוש באפיק תקשורת ה-SPI הטורי, הדורש קווי אות ייעודיים כגון קו שעון (SCK), קווי נתונים (MOSI ו-MISO), ופיני ברירת שבב נפרדים (CS) עבור המסך ועבור כרטיס ה-SD. הרכיב מתחבר ישירות לפיני ה-GPIO של בקר ה-ESP32, ומתוכנת באמצעות ספריות קוד מתקדמות (כמו TFT_eSPI או Adafruit_GFX) המאפשרות יצירת ממשק משתמש (GUI) אינטראקטיבי ומקצועי.

# מרכב הרובוט, מערכת הנעה ואנקודרים אופטיים ( Robot Chassis, DC Motors with Wheel Encoders)

רכיב זה הוא התשתית המכנית הנושאת של הרובוט האוטונומי, ומספק לו את יכולת התנועה המבוקרת במרחב ואת היכולת למדוד מרחקים נסיעה מדויקים לצוך חישוב אורך הקירות.



## תיאור המערכת המכנית ואופן פעולתה:

המערכת מורכבת משלושה תתי-רכיבים אינטגרליים הפועלים בסינרגיה:

1. שלדת הרובוט (Chassis): פלטת בסיס קשיחה המיוצרת מחומר אקרילי שקוף (פרספקס) או סיבי פחמן, הכוללת חריצי ומפתחי קיבוע ייעודיים. השלדה משמשת כפלטפורמה המרכזית שעליה מותקנים פיזית כל רכיבי החומרה: בקר ה-ESP32, לוח ההרחבה, מערך החיישנים, הצג ומקור הזינה (בית הסוללות). היא מעניקה לרובוט יציבות מכנית ומגנה על הרכיבים האלקטרוניים במהלך הנסיעה.
2. מנועי זרם ישר (DC Motors) ותמסורת: הרובוט מונע באמצעות שני מנועי גיר ישר-זווית המחוברים לגלגלים הראשיים. מנועים אלו ממירים אנרגיה חשמלית המגיעה מדרייבר המנועים (DRV8833) לאנרגיה מכנית סיבובית. תיבת הגיר (התמסורת) המובנית בכל מנוע מפחיתה את מהירות הסיבוב של המנוע ומגדילה את המומנט (כוח הסיבוב), מה שמאפשר לרובוט לנוע ביציבות, במהירות קבועה ובהתגברות על חיכוך הריצוף. בחלקו האחורי או הקדמי מותקן גלגל חופשי (Castor Wheel) המאפשר היגוי וסיבוב גמיש של הרובוט בפינות.
3. אנקודרים אופטיים (Optical Encoders): על גבי ציר המנוע מותקנת דיסקית מחורצת פלסטית המסתובבת יחד עם הגלגל. הדיסקית ממוקמת בתוך חיישן אופטי בצורת פרסה המכיל דיודת פולטת אור (IR LED) מצד אחד ופוטו-טרנזיסטור קולט מהצד השני. כאשר הדיסקית מסתובבת, השיניים שלה קוטעות את קרן האור לסירוגין. תהליך זה מייצר פולסים חשמליים ריבועיים (אות דיגיטלי) המועברים ישירות לבקר ה-ESP32 דרך פיני פסיקה (Interrupts).

### אופן החישוב המדעי:

על ידי ספירת כמות הפולסים (N) המתקבלים מהחיישן, הבקר מחשב את המרחק שהרובוט עבר במדויק לפי הנוסחה הגיאומטרית הבאה:

$$\text{Distance} = \frac{N}{\text{CPR}} \times (\pi \times D)$$

כאשר:

D - קוטר הגלגל (במילימטרים).

CPR - כמות החריצים בדיסקית (רזולוציית החיישן לסיבוב מלא).

$\pi$ - הקבוע המתמטי (3.1415).

### החלק באלגוריתם של הפרויקט:

בזמן שהרובוט נע לאורך הקיר, הבקר משווה באופן רציף את קצב הפולסים המגיע מהאנקודר של גלגל ימין לעומת גלגל שמאל. אם מתגלה הבדל במהירויות (לדוגמא עקב החלקה או שוני מכני קל במנועים), הבקר מבצע תיקון אקטיבי לאות ה-PWM הנשלח לדרייבר המנועים כדי לשמור על קו נסיעה ישר לחלוטין במקביל לקיר. מרגע הפעלת הרובוט בפינה, הבקר מתחיל לצבור את הפולסים. עם הגעה לפינה הבאה וזיהוי קיר חוסם, הבקר עוצר את המנועים, מתרגם את הפולסים שנצברו למרחק במטרים, ושומר את הערך כ- "אורך קיר X". בזמן פנייה בפינות החדר, המנועים פועלים בכיוונים מנוגדים (מנוע אחד קדימה ומנוע שני אחורה). הבקר משתמש בשילוב של נתוני האנקודרים וחיישן ה-IMU כדי לעצור את הסיבוב בדיוק ברגע שהרובוט השלים פנייה מדויקת לקיר הבא.

## תיאור הארכיטקטורה:

המערכת שלי בנויה בצורה שמאפשרת זרימת מידע רציפה מהשטח ועד למשתמש הקצה, כשהכל מנוהל על ידי הבקר המרכזי. אפשר לחלק את הארכיטקטורה לשלושה חלקים עיקריים שעובדים יחד:

החלק הראשון הוא מערך החיישנים והמנועים שמותקנים על הרובוט. הם "העיניים והרגליים" של המערכת. הם אוספים את כל הנתונים מהחדר – מרחקים, מהירות תנועה ומיקום – ומעבירים אותם לבקר המרכזי באמצעות תקשורת דיגיטלית מהירה (כמו I2C למשל, בשביל החיישנים).

החלק השני הוא הלוגיקה שרצה בתוך הבקר עצמו. פה בעצם קורה הקסם: הבקר לוקח את כל הנתונים הגולמיים מהחיישנים, מעבד אותם בזמן אמת, ומחליט איזה פקודות לתת למנועים כדי שהרובוט ימשיך לנסוע צמוד לקיר בצורה בטוחה. בלי החלק הזה, הרובוט היה סתם אוסף רכיבים; פה בעצם נכנס כל האלגוריתם שכתבתי ששומר על הרובוט יציב ומדויק.

החלק השלישי הוא הממשק האלחוטי. בסוף המדידה, כשהנתונים כבר מעובדים ומוכנים, הבקר משתמש ב-Wi-Fi כדי לדחוף את כל המידע החוצה לאתר שבניתי. האתר הזה הוא בעצם ה"תחנה האחרונה" – שם המשתמש רואה את השרטוט המדויק ומוריד את הקבצים שהוא צריך לעבודה שלו.

# תיאור פרוטוקולי תקשורת:

כדי שהרובוט יפעל כמערכת אחת מלוכדת, הרכיבים השונים צריכים לתקשר עם בקר ה- ESP32 בשפה שהם מדברים במקור. בתכנון המערכת הקפדתי על התאמת פרוטוקול התקשורת לכל רכיב בהתאם למפרט הטכני שלו, במטרה לשמור על יציבות ומהירות עבודה אופטימלית בשטח.

## פרוטוקול I2C: הלב של איסוף הנתונים

עבור חיישן הלייזר (VL53L1X), חיישן ה-IMU ושעון ה-RTC, המערכת משתמשת בפרוטוקול I2C. זהו פרוטוקול המבוסס על שני קווי תקשורת בלבד (SDA ו-SCL). בחרתי לרכז רכיבים אלו על אפיק זה בשל העובדה שהם תומכים בכתובות ייחודיות המאפשרות שיתוף של אותו קו תקשורת, מה שמוביל לארכיטקטורה נקייה, חיסכון בפיני GPIO בבקר, ועמידות גבוהה בתנאי עבודה בשטח.

## פרוטוקול SPI: המהירות לממשק המשתמש

עבור המסך וה-SD Card, פרוטוקול I2C לא מספק את רוחב הפס הנדרש. לכן, נעשה שימוש בפרוטוקול SPI המובנה ברכיבים אלו. פרוטוקול זה מאפשר העברה מהירה של מידע גרפי וקצבי כתיבה גבוהים לכרטיס הזיכרון, וזה מונע השהיות (Latency) ומבטיח שהמשתמש יראה את תהליך המדידה בזמן אמת בצורה חלקה וברורה.

## חיבורי GPIO: בקרה ישירה ופשוטה

עבור רכיבים הדורשים הפעלה דיגיטלית פשוטה ללא צורך בהעברת נתונים מורכבת (כמו חיישני קירבה דיגיטליים ובאזר), נעשה שימוש ישיר בפיני ה-GPIO. גישה זו מביאה תגובה מיידית ("זמן אמת") ללא תיווך של פרוטוקול מורכב, מה שקריטי להתרעות בטיחות או לזיהוי מכשולים מהיר של הרובוט.

## תקשורת אלחוטית (Wi-Fi) ופרוטוקול HTTP:

לשלב הוצאת הנתונים השתמשתי ביכולות ה-Wi-Fi המובנות של ה-ESP32 בשילוב פרוטוקול HTTP. לאחר סיום המדידה, הרובוט פותח תקשורת מול השרת ושולח אליו את הנתונים כקובץ. מעבר לנוחות למשתמש, שיטה זו מאפשרת זרימת עבודה מודרנית: המידע נגיש באתר המערכת להורדה מיידית, וכך הרובוט מתממשק בקלות עם עולם התכנון הממוחשב.

# הסבר על הספריות:

## ספריית Adafruit_VL53L1X (חיישן הלייזר ToF)

הספרייה הזו היא אחת החשובות ביותר בפרויקט, מכיוון שהיא ה "חוש הראייה" של הרובוט. ללא ספרייה זו, עבודה מול חיישן הלייזר הייתה כרוכה בכתיבת קוד מורכב לתקשורת ישירה מול אוגרי החומרה של הרכיב (Register level programming) – משימה מורכבת שחשופה להמון טעויות.

ייעוד מרכזי: הספרייה מטפלת בכל המורכבות של מדידת זמן החזר הפוטונים ( Time of Flight). היא מתרגמת את האותות החשמליים לערך דיגיטלי במילימטרים ומאפשרת לי לקבל את המרחק מהקיר בצורה מדויקת ורציפה, מה שקריטי למיפוי של החדר.

פונקציות עיקריות:

setupLaserSensor(): הפונקציה שכתבתי כדי לאתחל את החיישן. היא מגדירה את ה-Timing Budget (למהירות דגימה) ומפעילה את הטווח (startRanging).

readLaserDistance(): זו הפונקציה המרכזית שאני כתבתי בקוד. היא מבצעת לולאת בדיקה של 10 ניסיונות כדי לוודא שהחיישן מוכן, ורק אז שולבת את הפקודה vl53.distance() כדי לקבל את המרחק בפועל. במידה והנתון לא זמין, היא מחזירה ערך שגיאה (-2), מה שמונע מהרובוט להסתמך על נתונים שגויים.

אתגרים ותקלות: בתחילת הדרך נתקלתי בבעיה שהחיישן החזיר לעיתים ערכים לא יציבים, במיוחד כשהרובוט היה רחוק מהקיר. כדי לפתור את זה, הוספתי בתוך הקוד שלי (בפונקציה loop) לוגיקה של סינון ריצודים: אני משווה את המרחק הנוכחי לזה שהתקבל לפניו (lastLaserDistance), ורק אם ההפרש גדול מ-5 מ"מ, אני מעדכנת את התצוגה. זה הפך את המדידה להרבה יותר חלקה ואמינה בזמן נסיעה.

## ספריות Adafruit_GFX ו-Adafruit_ST7796S (הממשק הגרפי)

כשבחרתי להוסיף מסך צבעוני לרובוט, רציתי שהוא יתפקד כמכשיר מדידה מקצועי ולא רק כ"מחשבון" בסיסי. הספריות האלו נבחרו כי הן מאפשרות לי שליטה מלאה על התצוגה, מבלי שאצטרך לכתוב פקודות ברמה נמוכה מדי (כמו הגדרת כל פיקסל בנפרד).

ייעוד מרכזי: ספריית ה-ST7796S מתפקדת כדרייבר התקשורת שמעביר את המידע מה-ESP32 למסך הספציפי שלי. ספריית ה-GFX היא המנוע הגרפי – היא מאפשרת לי לעצב טקסט, צבעים וצורות, ובכך ליצור ממשק ידידותי למשתמש בשטח.

פונקציות עיקריות:

setupDisplay(): הפונקציה שאני כתבתי לאתחול המסך – היא מגדירה את האוריינטציה (Rotation), מנקה את המסך ומציגה את כותרת המערכת.

updateLaserDisplay() / updateProximityDisplay(): אלו פונקציות מותאמות אישית שכתבתי כדי להציג נתונים בזמן אמת. הן משתמשות ב-fillRect כדי למחוק את המספר הישן לפני הצגת החדש, וב-setCursor + print כדי להציג את הערכים העדכניים למשתמש.

אתגרים ותקלות: בשלב הפיתוח נתקלתי בבעיה שהרובוט הגיב באיטיות, במיוחד בנסיעה. אחרי בדיקה, הבנתי שהמסך "שתה" יותר מדי משאבי עיבוד בגלל שרעננתי אותו בכל loop מהיר. ומה שעשיתי זה שיצרתי לוגיקה שמעדכנת את המסך רק כשהנתון (כמו מרחק הלייזר) משתנה משמעותית (מעל 5 מ"מ) או כשהשעון מתעדכן. זה הוריד את העומס מהמעבד באופן מיידי והחזיר לרובוט את המהירות והזרימה שלו.

## ספריות ה-WiFi ו-WebServer (הקישוריות האלחוטית)

בלי הספריות האלו, הרובוט היה רק מכשיר מדידה מקומי ששומר נתונים על כרטיס זיכרון. בזכותן, הפרויקט שלי הופך למערכת IoT (אינטרנט של דברים) חכמה, המאפשרת ממשק שליטה ובקרה אלחוטי מרחוק באמצעות דפדפן.

ייעוד מרכזי: הספריות האלו מאפשרות ל-ESP32 לפעול כנקודת גישה (Access Point). במקום שהרובוט יצטרך להתחבר לראוטר חיצוני, הוא משדר רשת Wi-Fi משל עצמו. ה-WebServer מתפקד כ"מלצרית" דיגיטלית – הוא מאזין לבקשות שמגיעות מהדפדפן של המשתמש ומחזיר בתגובה את דפי האינטרנט או הנתונים שביקשנו.

פונקציות עיקריות:

setupGUI(): פונקציה שכתבתי שמאחדת את כל התהליך: היא יוצרת את הרשת בעזרת WiFi.softAP (עם שם וסיסמה קבועים), ומגדירה בעזרת server.on את הנתיבים לאתר (כמו / לדף הבית ו-/start להפעלת המדידה).

runGUI(): פונקציה שרצה בתוך ה-loop הראשי וקוראת ל-server.handleClient(). זה מבטיח שהרובוט "מקשיב" כל הזמן לבקשות מהמשתמש בלי לעצור את שאר הפעולות (כמו קריאת חיישנים).

handleRoot(): זו הפונקציה שמעצבת את דף האינטרנט שנפתח בטלפון של המשתמש (כולל עיצוב ה-CSS), מה שמאפשר לי לשלוט במראה הממשק מרחוק.

אתגרים ותקלות: אתגר משמעותי היה להבטיח שהרובוט לא "יינעל" אם התקשורת מתנתקת. בהתחלה, ברגע שהיה איבוד תקשורת, השרת היה קורס. כדי לפתור את זה, הבנתי שהשרת צריך לרוץ בצורה אסינכרונית בתוך ה-loop, מה שמאפשר לרובוט להמשיך למדוד את החדר גם אם אין כרגע משתמש מחובר לאתר. זה למעשה מה שנותן לרובוט את היכולת להיות אוטונומי באמת – הוא לא תלוי בחיבור אינטרנט כדי להמשיך בעבודה שלו.

## ספריות Adafruit_MPU6050 ו-Adafruit_Sensor (חיישן תנועה)

בכל פעם שהרובוט מבצע פנייה, הוא צריך לדעת בדיוק כמה הוא הסתובב כדי לא לסטות מהמסלול. כאן נכנס לפעולה חיישן ה-IMU. הספריות האלו הן ה"מצפן" של הרובוט, והן מתרגמות את התנועה הפיזית לערכים דיגיטליים שהבקר יכול לעבד.

ייעוד מרכזי: הספרייה מנהלת את התקשורת המורכבת ב-I2C מול רכיב ה-MPU6050. היא לוקחת את הנתונים הגולמיים מהג'ירוסקופ ומד התאוצה ומתרגמת אותם לערכים הנדסיים. בלעדיה, הייתי צריכה לכתוב קוד ארוך ומורכב מאוד לניהול אוגרי החומרה והמרה של נתוני החיישן, משימה שקל מאוד לטעות בה.

פונקציות עיקריות:

setupMotion(): הפונקציה שכתבתי לאתחול החיישן. היא מגדירה את טווח התאוצה והג'ירוסקופ, ומשתמשת בפקודות begin כדי להעיר את החיישן ממצב שינה ולהכין אותו לעבודה.

getTiltX(): זו הפונקציה המרכזית שכתבתי בקוד. היא מבצעת קריאה לחיישן (mpu.getEvent) ושולפת מתוך המידע את ערך התאוצה בציר ה-X, מה שמאפשר לי לדעת בזמן אמת אם הרובוט נוטה או נמצא בשיפוע.

אתגרים ותקלות: האתגר הגדול היה ה"רעש" בקריאות החיישן. כשהמנועים עבדו, הוויברציות של הרובוט גרמו לחיישן להשתגע. הפתרון: למדתי להשתמש בפונקציה של הספרייה שנקראת setFilterBandwidth (MPU6050_BAND_21_HZ). זהו בעצם "פילטר דיגיטלי" (Low Pass Filter) מובנה בספרייה שמנקה את התדרים הגבוהים של הרעידות, וזה מה שגרם לנתונים להפוך ליציבים ואמינים גם בזמן שהרובוט נוסע על הרצפה.

## ספריית RTClib.h (שעון זמן אמת)

שעון זמן אמת (RTC) הוא מרכיב חשוב ברובוט המדידה שלי. כדי שהאדריכל שמשתמש ברובוט יוכל לנהל את העבודה שלו בצורה מסודרת, לכל מדידה צריכה להיות "חותמת זמן" מדויקת. ה-RTC נותן לרובוט זיכרון זמן קבוע, גם כשהוא מכובה ומנותק מהחשמל.

ייעוד מרכזי: הספרייה מאפשרת תקשורת I2C מול רכיב ה-DS3231. בעזרתה אני מושכת את הזמן העדכני בכל רגע נתון ומשלבת אותו בתצוגה על המסך ובנתונים שנשלחים לאתר האינטרנט. זה מאפשר למשתמש לדעת בדיוק מתי בוצעה כל סריקת חדר.

פונקציות עיקריות:

setupClock(): הפונקציה שכתבתי לאתחול השעון על פיני Wire1 (16, 17). הוספתי בה את הפקודה rtc.lostPower() שבודקת בכל הפעלה האם השעון איבד את הזיכרון שלו, ואם כן – היא מעדכנת אותו אוטומטית לפי זמן ההידור של הקוד (rtc.adjust).

getTimeString(): פונקציה מותאמת אישית שכתבתי, שמושכת את הנתונים מהשעון (rtc.now()) ומעצבת אותם למחרוזת טקסט נוחה להצגה על המסך (פורמט HH:MM).

אתגרים ותקלות: בגלל שהשתמשתי ביציאת Wire1 (פינים 16, 17) ולא ביציאה הראשית, בהתחלה לא הצלחתי לתקשר עם השעון והקוד החזיר ערכים שגויים או את תאריך ברירת המחדל (שנת 1970). אחרי בדיקת חיבורים, הבנתי שהסוללה הפיזית על הרכיב לא ישבה טוב בתושבת. ברגע שסידרתי את המגעים ושיניתי את ההגדרה בתוכנה לעבודה מול Wire1, הספרייה התחילה לעבוד בצורה יציבה והזמן מוצג עכשיו בדיוק מוחלט על מסך הרובוט.

## ניהול הנעת הרובוט (Motor Control)

כדי לא להעמיס על הקוד הראשי פקודות חשמליות מסובכות לדרייבר המנועים, ריכזתי את כל הלוגיקה של התנועה תחת פונקציות מסודרות. במקום לשלוט בפינים ישירות בכל פעם, יצרתי שכבת "ניהול הנעה" שמאפשרת לי לשלוט ברובוט בצורה לוגית ופשוטה.

ייעוד מרכזי: המערכת מנהלת את הדרייבר (TB6612FNG) ואחראית על התרגום בין פקודות תנועה (כמו "קדימה", "אחורה" או "עצור") לבין שינויי מתח בפיני ה-GPIO. זה מאפשר לי לנהל את נסיעת הרובוט בתוך פקודה אחת פשוטה ב-loop, מה שהופך את הקוד לקריא ומסודר מאוד.

פונקציות עיקריות:

setupMotors(): פונקציה שאני כתבתי להגדרת הפינים של הדרייבר (OUTPUT) והבאת המנועים למצב עצירה בטוח (stopMotors) מיד עם הדלקת המערכת.

moveBackward / ()moveForward(): אלו הפונקציות שכתבתי כדי לקבוע את כיוון הנסיעה. הן שולטות בפיני ה-AIN1/2 וה-BIN1/2 בצורה מסונכרנת, כך ששני המנועים יקבלו את הפקודה הנכונה בו-זמנית.

stopMotors(): פונקציה קריטית שמוודאת שכל הפינים של הדרייבר נכבים (LOW), מה שמונע מהמנועים להמשיך להסתובב כשצריך לעצור (במיוחד בזמן עבודה אוטומטית).

אתגרים ותקלות: האתגר הכי משמעותי היה העובדה שהמנועים לא נעו באותו קצב, מה שגרם לרובוט לסטות מהמסלול ולנסוע בעיקול במקום בקו ישר. הפתרון: למדתי שאני לא יכולה להסתמך על זה שהמנועים "זהים". לכן, בניתי לוגיקה פשוטה בתוך הקוד שמאפשרת לי "לאזן" את המנועים. אם הייתי צריכה, הייתי מוסיפה תיקון קטן במהירות, ובסוף הגעתי למצב שהרובוט נוסע בקו ישר ומדויק. זה לימד אותי המון על ההבדל שבין "תיאוריה" (קוד זהה לשני הצדדים) לבין ה"מציאות" (הבדלים פיזיקליים בין מנועים).

## ספריית הטיפול בנתונים (Data Handling)

זו "המוח" של המערכת. התפקיד שלה הוא לקחת את כל הנתונים הגולמיים מהחיישנים (מרחק, זווית, זמן) ולהפוך אותם למידע שהמשתמש יכול לראות ולהבין דרך האתר. בלי החלק הזה, כל הנתונים היו נשארים בתוך הבקר כערכים מספריים בלבד.

ייעוד מרכזי: הספרייה (או חטיבת הקוד הזו) אחראית על ניקוי הנתונים מהחיישנים – למשל, סינון קריאות "Too Far" או קריאות לא הגיוניות של הלייזר – והצגתם בצורה מאורגנת למשתמש. היא זו שמאפשרת לי להעביר את הנתונים מהרובוט לדפדפן בצורה מובנית.

פונקציות עיקריות:

handleRoot(): זו הפונקציה המרכזית בשרת. היא בונה את דף ה-HTML בזמן אמת, כולל את כל עיצוב ה-CSS והטקסט שהמשתמש רואה בדפדפן, ובכך היא "אורזת" את המידע בצורה שנוחה לקריאה.

updateClockDisplay / ()updateLaserDisplay(): אלו פונקציות שבודקות את הנתונים לפני שהם מוצגים. הן מוודאות שאם קיבלתי ערך שגיאה (כמו -2 או -1), המסך יציג למשתמש הודעה ברורה ("Too Far") במקום להציג מספרים שגויים.

אתגרים ותקלות: האתגר הגדול היה לנהל את הנתונים בתוך זיכרון ה-ESP32 בלי שהוא ייתקע. בהתחלה, שלחתי דפי אינטרנט ענקיים וטקסטים ארוכים וזה גרם ל"זליגת זיכרון"

(Memory Leak). אז למדתי לעבוד עם מחרוזות בצורה חסכונית ולנקות את הבאפר של השרת אחרי כל שליחה. זה מה שמאפשר לרובוט להמשיך לרוץ שעות בלי לעשות Reset מעצמו.

# תהליך הפיתוח:

## אתגרים ותקלות בתהליך הפיתוח

במהלך העבודה על הפרויקט נתקלתי בכמה קשיים טכניים משמעותיים. חלקם נפתרו בדרכים יצירתיות וחלקם עדיין מהווים אתגר למידה שאני ממשיכה לחקור.

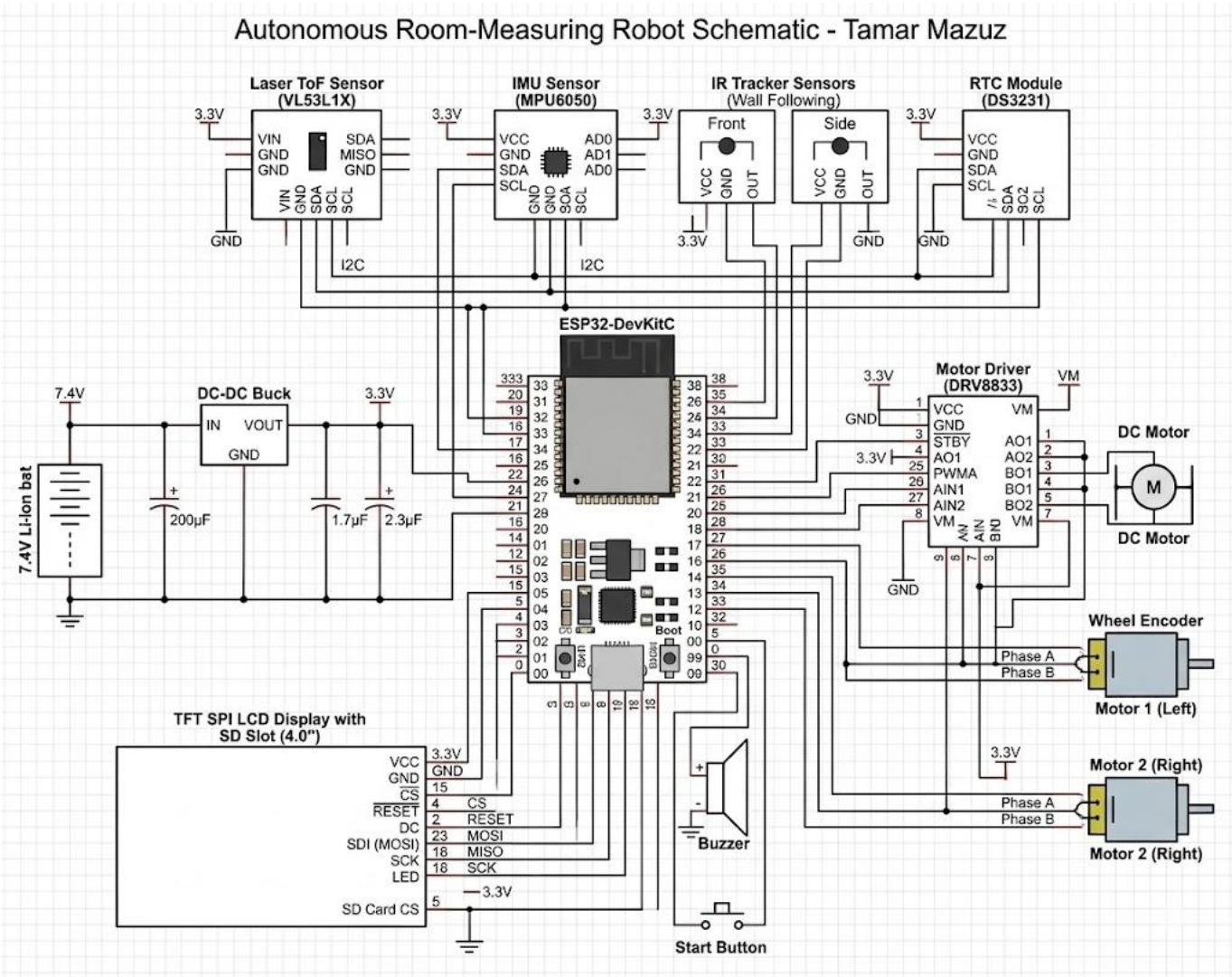
**הבעיה בתנועת המנועים (הגלגלים מסתובבים לכיוונים מנוגדים):** אחד האתגרים שעדיין מלווים אותי הוא הסנכרון של המנועים. בשלבי החיבורים, ראיתי שהגלגלים לפעמים מסתובבים לכיוונים מנוגדים במקום לדחוף את הרובוט קדימה. זה קורה כי המנועים מחוברים הפוך מבחינת הקוטביות (Polarity). בגלל שזה רכיב פיזי, הדרך הכי פשוטה לפתור את זה היא להחליף את החיבור של החוטים בדרייבר (שזה לא עזר לי..), או לחלופין – מה שאני מנסה לעשות עכשיו – זה לשנות את הלוגיקה בקוד כך שכל מנוע יקבל פקודת כיוון שונה. זה לימד אותי המון על איך מנועי DC עובדים ועל החשיבות של חיבור נכון לפי השרטוט החשמלי.

**האתגר עם המסך (המסך הלבן):** בהתחלה המסך סירב להידלק והראה רק מסך לבן. זו הייתה אחת התקלות הכי מתסכלות שדרשה ממני שעות של עבודה. אחרי המון ניסיונות הבנתי שזה קשור לתיאום בתוך ספריית ה- TFT הייתי צריכה להגדיר נכון את הפינים של ה-Reset וה-CS בקוד כדי שהבקר ידע "להעיר" את המסך בזמן. ברגע שהבנתי שהבעיה היא בתקשורת ה-SPI ושצריך להמתין שבריר שנייה שהמסך יאתחל לפני ששולחים פקודות גרפיות – הוא סוף סוף נדלק.

**בעיית איבוד זמן בשעון ה-RTC:** השעון בזמן אמת (RTC) עשה לי בעיות בתחילת הדרך. בגלל שהסוללה הקטנה שעל הרכיב לא תמיד ישבה טוב במקום, השעון היה מתאפס בכל פעם שהייתי מנתקת את הרובוט מהחשמל. למדתי שזה קורה בגלל מגעים רופפים בתושבת הסוללה. החלפתי את התושבת וקיבעתי את הסוללה בצורה שלא תזוז. מעבר לזה, הוספתי בקוד פונקציה שבודקת בכל הפעלה האם הזמן שנמשך מהשעון הוא הגיוני (למשל, שזה לא שנת 2000). אם השעון לא תקין, המערכת מוציאה התראה במסך. זה הוסיף רובד של אמינות לתיעוד המדידות שלי.

# תיעוד הפתרון:

## שרטוט חשמלי:



## חיבורים ואופן הפעולה:

בפרק זה אפרט את מערך החיבורים בין רכיבי החומרה לבקר ה-ESP32. כל הרכיבים פועלים במתח של V3.3, המוסדר דרך מייצב מתח (Buck Converter) המקבל הזנה מסוללת V7.4.

## טבלת חיבורים:

| רכיב | פין ברכיב | חיבור לבקר/פס מתח | תפקיד |
|---|---|---|---|
| ESP32 | 3.3V | פס מתח אדום | הזנת מתח למערכת |
| ESP32 | GND | פס מתח כחול | הארקה משותפת |
| MPU6050 | GND / VCC | 3.3V / GND | הזנה |
| MPU6050 | SDA / SCL | פינים 16 /17 | תקשורת I2C |
| Laser ToF | VCC / GND | 3.3V / GND | הזנה |
| Laser ToF | SDA / SCL | פינים 21 / 22 | תקשורת I2C |
| Motor Driver | VCC / GND | 3.3V / GND | הזנה |
| Motor Driver | PWMA / AIN1 / AIN2 | פינים 25 / 26 / 27 | בקרת ומהירות |
| Motor Driver | STBY | 3.3V | הפעלת הדרייבר |
| Buzzer | + / - | פין 27 / GND | התראות קוליות |
| TFT LCD | VCC / GND | 3.3V / GND | הזנה |
| TFT LCD | CS / RST / DC | פינים 2 / 4 / 15 | בקרה |
| TFT LCD | SDI / SCK | פינים 18 / 23 | תקשורת SPI |

## פירוט חיבורי החומרה ואופן הפעולה

**MPU-6050 - חיישן תנועה (IMU)**

חיישן ה-MPU6050 מספק נתוני תאוצה וסיבוב (Gyroscope) לבקר. החיישן עובד בתקשורת I2C, המאפשרת לו להעביר את נתוני התנועה בצורה דיגיטלית ישירה. החיישן מוזן ממתח V3.3 כדי למנוע נזק לרכיבי הבקר.

VCC → 3.3V (פס מתח)

GND → GND (פס מתח)

SDA → פין 16 ב-ESP32

SCL → פין 17 ב-ESP32

**חיישן מרחק לייזר (Laser ToF)**

חיישן ה-ToF (Time of Flight) מבצע מדידת מרחקים מדויקת באמצעות החזר קרני לייזר. הוא מחובר גם כן על אפיק I2C, מה שמאפשר לבקר לתקשר איתו במקביל לחיישנים אחרים.

VCC → 3.3V (פס מתח)

GND → GND (פס מתח)

SDA → פין 21 ב-ESP32

SCL → פין 22 ב-ESP32

**מסך מגע 4 אינץ' (TFT LCD)**

המסך משמש להצגת נתונים גרפיים בזמן אמת. הוא משתמש בממשק SPI המאפשר קצב רענון גבוה, הנדרש להצגת גרפיקה מורכבת. בנוסף, פיני ה-Touch (T_CS, T_IRQ) מאפשרים למשתמש אינטראקציה עם הרובוט.

VCC → 3.3V | GND → GND

CS → פין 15 | RST → פין 4 | DC → פין 2

SDI (MOSI) → פין 23 | SCK → פין 18

T_CS → פין 21 | T_IRQ → פין 22

**דרייבר מנועים (TB6612FNG)**

הדרייבר אחראי על תרגום פקודות הבקר לתנועה פיזית של המנועים. הוא מקבל אותות PWM לשליטה במהירות ואותות GPIO לקביעת כיוון הסיבוב. הוא מחובר לסוללת V7.4 (דרך פין VM) כדי לספק למנועים את הכוח הדרוש, בעוד הלוגיקה של הדרייבר מוזנת מ-V3.3 מהבקר.

VCC → 3.3V | GND → GND

PWMA → פין 25 (בקרת מהירות)

AIN1 → פין 26 (כיוון 1)

AIN2 → פין 27 (כיוון 2)

STBY → 3.3V (מצב פעיל)

AO1/AO2 → חיבור ישיר למנוע DC

**באזר (Buzzer)**

הבאזר משמש להתראות קוליות (סיום מדידה, שגיאות). הוא מחובר לפין דיגיטלי ב-ESP32 ששולח פעימות מתח היוצרות צליל.

רגל ארוכה (+) → פין 27 ב-ESP32

רגל קצרה (-) → GND (פס מתח)

## קטעי קוד:

### :ESP32

```
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_ST7796S.h>
#include <Adafruit_VL53L1X.h>
#include <RTClib.h>
#include <Adafruit_MPU6050.h>
#include <Adafruit_Sensor.h>

const int buzzerPin = 27;
const int targetDistance = 200; // מרחק רצוי מהקיר במילימטרים (20 ס"מ)
const int tolerance = 30;       // טווח סטייה מותר (למניעת "דיגדוג" מיותר)

#define PROXIMITY_PIN 13
#define TFT_CS 15
#define TFT_DC 2
#define TFT_RST 4

Adafruit_ST7796S tft = Adafruit_ST7796S(TFT_CS, TFT_DC, TFT_RST);
Adafruit_VL53L1X vl53 = Adafruit_VL53L1X();
RTC_DS3231 rtc;
Adafruit_MPU6050 mpu;
int lastLaserDistance = 0;

void setupGUI();
void runGUI();

void setup() {
  Serial.begin(115200);
  pinMode(buzzerPin, OUTPUT);
  digitalWrite(buzzerPin, HIGH);
  delay(200);
  digitalWrite(buzzerPin, LOW);
  setupDisplay();
  setupProximitySensor();
  setupLaserSensor();
  setupClock();
  setupMotion();
  setupMotors();
  setupGUI();
  Serial.println("All systems initialized successfully!");
}
```

```
  int proxStateRight;
  int proxStateLeft;
  int proxStateForward;
  int proxStateBackward;

void loop() {
  server.handleClient();
  String currentTime = getTimeString();
  updateClockDisplay(currentTime);
  readProximity();
  updateProximityDisplay(proxState);
  int laserDistance = readLaserDistance();
  updateLaserDisplay(laserDistance);
  if (isScanning) {
    if (endScan()== true) {
      stopMotors();
      beepBuzzer();
    }
    else {
      if (proxStateRight == false) {
          turnRight();
        }
      else if ( proxStateForward == false) {
        moveForward();
      }
      else if (proxStateRight == true) {
          turnLeft();();
        }
      }
    }
  }
  float tilt = getTiltX();
}
```

```
float xTilt = getTiltX();
updateMotionDisplay(xTilt);

int proximityState = readProximity();
updateProximityDisplay(proximityState);

moveForward();

delay(10);
}
```

## Display:

```
void setupDisplay() {
  tft.init();
  tft.setRotation(1);
  tft.fillScreen(ST77XX_BLACK);
  tft.setTextColor(ST77XX_WHITE);
  tft.setTextSize(3);
  tft.setCursor(20, 50);
  tft.print("System Testing:");
}

void updateProximityDisplay(int state) {
  tft.fillRect(20, 100, 250, 30, ST77XX_BLACK);
  tft.setCursor(20, 100);
  tft.setTextSize(2);
  if (state == LOW) {
    tft.setTextColor(ST77XX_GREEN);
    tft.print("Prox: DETECTED!");
  } else {
    tft.setTextColor(ST77XX_RED);
    tft.print("Prox: CLEAR");
  }
}

void updateLaserDisplay(int distance) {
  tft.fillRect(20, 140, 250, 30, ST77XX_BLACK);
  tft.setCursor(20, 140);
  tft.setTextColor(ST77XX_CYAN);
  tft.setTextSize(2);
  if (distance == -1) {
    tft.print("Laser: Too Far");
  } else {
    tft.print("Laser: ");
    tft.print(distance);
    tft.print(" mm");
  }
}
```

```
void updateClockDisplay(String timeStr) {
  tft.fillRect(220, 10, 100, 30, ST77XX_BLACK);
  tft.setCursor(220, 10);
  tft.setTextColor(ST77XX_YELLOW);
  tft.setTextSize(2);
  tft.print(timeStr);
}

void updateMotionDisplay(float tilt) {
  tft.fillRect(20, 180, 250, 30, ST77XX_BLACK);
  tft.setCursor(20, 180);
  tft.setTextColor(ST77XX_WHITE);
  tft.setTextSize(2);
  tft.print("Tilt X: ");
  tft.print(tilt, 1);
}
```

## Motors:

```
const int motorAIN1 = 25;
const int motorAIN2 = 14;
const int motorBIN1 = 32;
const int motorBIN2 = 33;

void setupMotors() {
  pinMode(motorAIN1, OUTPUT);
  pinMode(motorAIN2, OUTPUT);
  pinMode(motorBIN1, OUTPUT);
  pinMode(motorBIN2, OUTPUT);

  stopMotors();
}

// נסיעה קדימה - שני הצדדים מסתובבים קדימה
void moveForward() {
  digitalWrite(motorAIN1, HIGH);
  digitalWrite(motorAIN2, LOW);
  digitalWrite(motorBIN1, HIGH);
  digitalWrite(motorBIN2, LOW);
}

// נסיעה אחורה - שני הצדדים הופכים כיוון
void moveBackward() {
  digitalWrite(motorAIN1, LOW);
  digitalWrite(motorAIN2, HIGH);
  digitalWrite(motorBIN1, LOW);
  digitalWrite(motorBIN2, HIGH);
}

// עצירה מוחלטת לכל 4 הגלגלים
void stopMotors() {
  digitalWrite(motorAIN1, LOW);
  digitalWrite(motorAIN2, LOW);
  digitalWrite(motorBIN1, LOW);
  digitalWrite(motorBIN2, LOW);
}
```

```
// פנייה קלה ימינה - מנוע שמאל מהיר יותר או מנוע ימין איטי
void turnRight() {
  digitalWrite(motorAIN1, HIGH); // מנוע ימין עוצר
  digitalWrite(motorAIN2, HIGH);
  digitalWrite(motorBIN1, HIGH); // מנוע שמאל נוסע
  digitalWrite(motorBIN2, LOW);
}

// פנייה קלה שמאלה
void turnLeft() {
  digitalWrite(motorAIN1, HIGH);
  digitalWrite(motorAIN2, LOW);
  digitalWrite(motorBIN1, HIGH); // מנוע שמאל עוצר
  digitalWrite(motorBIN2, HIGH);
}
```

:Clock

```
void setupClock() {
  Wire1.begin(16, 17);
  if (!rtc.begin(&Wire1)) {
    Serial.println("Couldn't find RTC");
  } else {
    if (rtc.lostPower()) {
      Serial.println("RTC lost power, lets set the time!");
      rtc.adjust(DateTime(F(__DATE__), F(__TIME__)));
    }
    Serial.println("Clock Initialized!");
  }
}

String getTimeString() {
  DateTime now = rtc.now();
  String hour = String(now.hour());
  String minute = String(now.minute());
  if (now.hour() < 10) hour = "0" + hour;
  if (now.minute() < 10) minute = "0" + minute;
  return hour + ":" + minute;
}
```

:Motion

```
void setupMotion() {
  if (!mpu.begin(0x68, &Wire1)) {
    Serial.println("Failed to find MPU6050 chip");
  } else {
    Serial.println("MPU6050 Found!");
    mpu.setAccelerometerRange(MPU6050_RANGE_8_G);
    mpu.setGyroRange(MPU6050_RANGE_500_DEG);
    mpu.setFilterBandwidth(MPU6050_BAND_21_HZ);

    Wire1.beginTransmission(0x68);
    Wire1.write(0x6B);
    Wire1.write(0x00);
    Wire1.endTransmission();
  }
}

float getTiltX() {
  sensors_event_t a, g, temp;
  mpu.getEvent(&a, &g, &temp);
  return a.acceleration.x;
}
```

## :Laser_sensor

```
void setupLaserSensor() {
  Wire.begin(21, 22);
  Wire.setClock(100000);
  if (!vl53.begin(0x29, &Wire)) {
    Serial.println("Laser VL53L1X not found!");
  } else {
    Serial.println("Laser Found!");
    vl53.setTimingBudget(50);
    vl53.startRanging();
  }
}

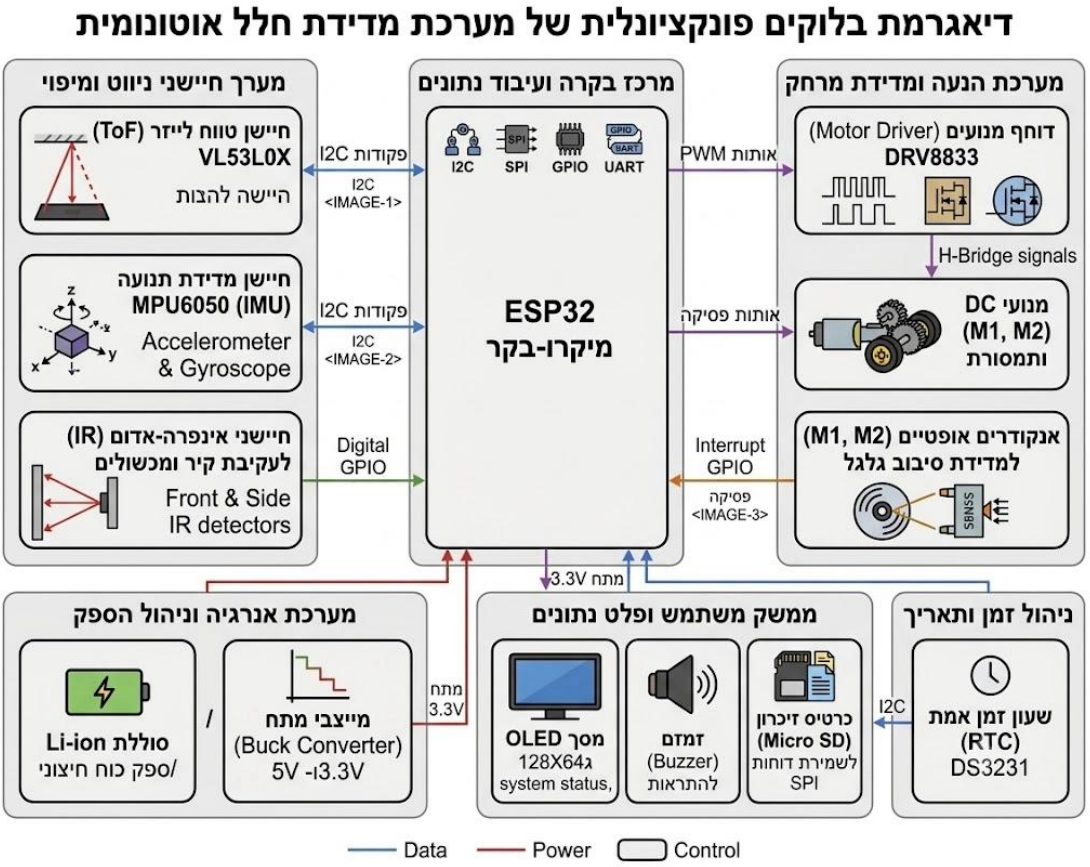
int readLaserDistance() {
  for (int i = 0; i < 10; i++) {
    if (vl53.dataReady()) {
      int distance = vl53.distance();
      vl53.clearInterrupt();
      return distance;
    }
    delay(5);
  }
  return -2;
}
```

## :ProximitySensor

```
void setupProximitySensor() {
  pinMode(PROXIMITY_PIN, INPUT);
}

int readProximity() {
  return digitalRead(PROXIMITY_PIN);
}
```

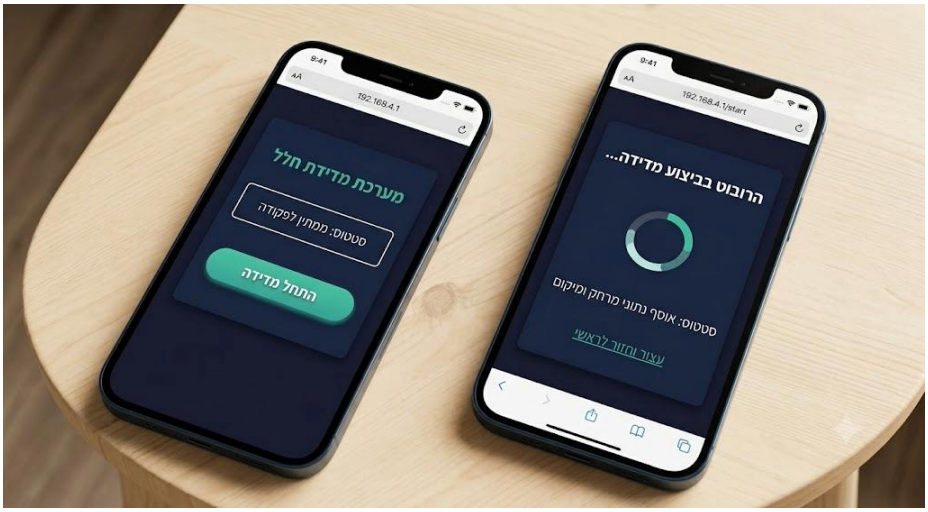
## דיאגרמת בלוקים:




## תרשים זרימה:



# :GUI




## מדריך למשתמש:

**ברוך הבא!**

מערכת זו נועדה לבצע מדידות חלל מדויקות באופן אוטונומי, תוך חיסכון בזמן ומניעת טעויות אנוש. המדריך הבא יסייע לך להפיק את המיטב מהמכשיר.

**אז איך מתחילים?**

הנח את המכשיר צמוד לקיר בנקודת ההתחלה הרצויה על גבי משטח ישר ויציב.

ניתן להפעיל את המכשיר בשתי דרכים:

1. לחיצה על כפתור ההפעלה הראשי בגוף המכשיר.
2. לחיצה ישירה על כפתור ה"התחל" המופיע על גבי מסך המגע של המכשיר.

המכשיר משדר רשת Wi-Fi פנימית. חבר את הטלפון או המחשב שלך לרשת המכשיר (שמה יופיע ברשימת הרשתות הזמינות).

לאחר החיבור, פתח את דפדפן האינטרנט והקלד את כתובת ה-IP המופיעה על צג המכשיר. דף הניהול יופיע מיד על המסך.

לאחר מכן לחץ על כפתור "התחל סריקה" (Start Scan) במסך. המכשיר יחל לנוע ולמפות את החלל באופן עצמאי.

מה קורה בזמן העבודה?

תוכל לעקוב אחר התקדמות המדידה דרך מסך הדפדפן בטלפון שלך.

המכשיר ישמיע צליל (באזר) במידה ונתקל במכשול או בסיום התהליך.

כל המידע נשמר בזמן אמת גם על כרטיס הזיכרון הפנימי, כך שאין חשש לאובדן נתונים גם במקומות ללא קליטה.

סיום התהליך:

עם סיום הסריקה, הנתונים המעובדים (שטח, נפח ומידות החלל) יופיעו על גבי מסך הניהול בדפדפן. תוכל להוריד אותם ישירות למחשב או לטלפון כקובץ מוכן לעבודה.

## פיתוחים עתידיים ושדרוגי מערכת:

תהליך פיתוח הרובוט היווה עבורי מקום נרחב ללמידה הנדסית, אך כל מוצר טכנולוגי אינו מסתיים לעולם, אלא תמיד מצוי בתהליך איטרטיבי של שדרוג ושיפור. להלן מספר כיוונים מרכזיים להמשך פיתוח המערכת:

בשלב הבא, אני מתכננת לשלב אלגוריתמים של למידת מכונה (Machine Learning) שיאפשרו לרובוט לא רק למדוד מרחק, אלא "להבין" מה הוא רואה. זיהוי אוטומטי של אלמנטים אדריכליים כמו דלתות, חלונות ורהיטים יאפשר לרובוט לייצר מפה "נקייה" של החלל. זהו שדרוג קריטי שיחסוך מהמשתמש את הצורך בפינוי הרהיטים מהחדר או בחישובים ידניים מורכבים הרובוט פשוט ידע להתעלם מהרהיטים ולהתמקד במעטפת הקירות.

בנוסף אחד היעדים המרכזיים שלי הוא סגירת המעגל המלא בין המדידה לתוכנית העבודה. במקום להסתפק בקובץ נתונים גולמי, המערכת תשודרג כך שבלחיצת כפתור אחת, הנתונים יומרו לקובץ DWG הפורמט הסטנדרטי של תוכנת AutoCAD. כך, האדריכל יקבל תוכנית מידות מוכנה להגשה ישירות מהרובוט, ללא צורך בהקלקת נתונים ידנית או בשרטוט נוסף.

הוספת חיישן "מגניב" - חיישן אינפרא- אדום תרמי כתוספת טכנולוגית מתקדמת, אני שואפת לשלב מצלמה תרמית קטנה. מעבר למדידת החלל, המצלמה תאפשר לרובוט לזהות מוקדי איבוד אנרגיה בבית (כמו דליפות חום מפתחים לא אטומים או רטיבות בקירות שלא נראית לעין). השילוב בין מדידה גיאומטרית לניתוח טרמי יהפוך את הרובוט מכלי מדידה לכלי אבחון אדריכלי מקיף.

**לסיכום:** הפרויקט הנוכחי הניח את התשתית הטכנולוגית למוצר שיכול לחסוך עשרות שעות עבודה בשבוע למשרדי תכנון. כל השיפורים המוצעים לעיל נועדו להפוך את הרובוט מכלי עזר לכלי מקצועי, יציב ואוטונומי לחלוטין, המספק מענה מדויק ומקיף לצרכי האדריכל המודרני.

## סיכום ומסקנות:

העבודה על פרויקט "רובוט המדידה האוטונומי" הייתה עבורי מסע הנדסי מאתגר ומרתק. כשהתחלתי את הפרויקט, המטרה שלי הייתה למצוא דרך טכנולוגית שתהפוך מדידות ידניות ומייגעות לתהליך חכם, מהיר ומדויק. היום, כשאני מביטה ברובוט המוכן, אני מרגישה שהצלחתי להגשים את הזה.

למדתי שבתכנון מערכת המשלבת חומרה ותוכנה, אין מקום לקיצורי דרך. כל חוט שלא היה מחובר נכון או קוד שלא עבר אופטימיזציה, השפיעו ישירות על יציבות הרובוט. הבנתי שסדר וארגון בבניית המעגלים ובכתיבת הקוד הם המפתח למערכת שעובדת ללא תקלות.

היו רגעים שבהם הקוד לא הגיב או שהחיישנים לא העבירו נתונים מדויקים. הלמידה הכי גדולה שלי הייתה היכולת "לעצור, לחשוב ולנתח". הבנתי שלא כל תקלה היא כישלון, אלא הזדמנות להבין עמוק יותר איך בקר ה- ESP32 והחיישנים מתקשרים ביניהם.

בתחילת הדרך התמקדתי רק ב"שזה יעבוד". לקראת הסוף הבנתי שרובוט הוא לא רק מנועים וקוד, אלא מוצר שמישהו צריך להפעיל. לכן, ההשקעה ב-GUI האינטרנטי ובפשטות התפעול הפכה את הפרויקט מ"פרויקט לימודים" ל"מוצר הנדסי."

לסיכום, הפרויקט הזה העניק לי ביטחון ביכולת שלי לקחת רעיון מדומיין ולהפוך אותו למציאות טכנולוגית עובדת. אני יוצאת מהפרויקט הזה לא רק עם ידע ב C++ ובאלקטרוניקה, אלא עם תפיסת עולם של פתרון בעיות – גישה שאקח איתי לכל פרויקט הנדסי בעתיד.

# ביבליוגרפיה:

https://www.st.com/en/imaging-and-photonics-solutions/vl53l0x.html

https://learn.adafruit.com/adafruit-gfx-graphics-library

https://docs.espressif.com/projects/esp-idf/en/v4.2.3/esp32/hw-reference/modules-and-boards.html

הסבר על בקר ESP32

https://www.elec4u.co.il/archives/7514

הסבר על חיישן MPU6050

https://www.elec4u.co.il/archives/6959

https://iw.ariat-tech.com/blog/optimizing-performance-with-mpu-6050-setup,calibration,and-applications.html

חיישן מרחק לייזר GY-VL53L1X

https://iw.hwlibre.com/%D7%94%D7%9B%D7%9C-%D7%A2%D7%9C-%D7%97%D7%99%D7%99%D7%A9%D7%9F-%D7%9E%D7%A8%D7%97%D7%A7-%D7%9C%D7%99%D7%99%D7%96%D7%A8-%D7%91%D7%A8%D7%9E%D7%AA-%D7%93%D7%99%D7%95%D7%A7-/%D7%92%D7%91%D7%95%D7%94%D7%94-vl53l0x

https://iw.allelcoelec.com/blog/VL53L1X-Laser-Ranging-Sensor-Specifications,Pinout-and-Circuit-Diagram.html

שעון זמן אמת RTC:

https://making-il.blogspot.com/2018/10/rtc.html

chrome-extension://efaidnbmnnnibpcajpcglclefindmkaj/https://www.arikporat.com/wp-content/uploads/2022/12/rtc_DS1307.pdf